Team ID: 26FSPDSCPMID4098

Case-Study Report

On "Resilient Data Science Programming for Fault-Tolerant Distributed Analytics in Multi-Cluster Spark Environments"

submitted as part of

Programming for Data Science (MACSE502)

by

26MML0054
Name

26MML0006
Name

26MML0047
Name

26MML0015
Name

To



Dr M Rajasekhara Babu

School of Computer Science and Engineering

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

Index

Sl. No.	Section Topic	Page No.
	Executive Summary and Abstract	7
1.	I. Introduction	7
2.	II. Problem Statement	8
3.	III. Background and Literature Review	9
4.	IV. Requirements Analysis for Resilient Analytics Systems	11
5.	V. Multi-Cluster Spark Analytics Architecture	13
6.	VI. Distributed Data Engineering Layer	14
7.	VII. Spark Processing and Execution Layer	15
8.	VIII. Fault Detection and Monitoring Framework	16
9.	IX. Fault-Tolerant Recovery and Resilience Layer	18
10.	X. Multi-Cluster Coordination and Orchestration Layer	20
11.	XI. Explainability and Reliability Assessment Layer	21
12.	XII. Security, Governance, and Compliance Layer	22
13.	XIII. Experimental Setup	23
14.	XIV. Evaluation Metrics	25
15.	XV. Results and Findings	25
16.	XVI. Comparative Analysis	28
17.	XVII. Reliability and Business Continuity Assessment	29
18.	XVIII. Industry Use Cases and Case Study Evaluation	30
19.	XIX. Business Impact Assessment	31
20.	XX. Challenges and Limitations	32
21.	XXI. Future Research Directions	32
22.	XXII. Conclusion	33
	References	34
	Appendix	34

List of Figures

Fig. No.	Figure Title	Page No.
1.	Evolution of Distributed Analytics Platforms	7
2.	Resilient Data Science Ecosystem	8
3.	Failure Scenarios in Distributed Analytics	8
4.	Literature Taxonomy Framework	9
5.	Requirements Engineering Framework	12
6.	Resilient Multi-Cluster Spark Architecture	13
7.	Layered Analytics Stack	14
8.	Distributed Data Engineering Pipeline	14
9.	Spark Execution Architecture	15
10.	DAG Processing Workflow	16
11.	Monitoring and Observability Architecture	17
12.	Fault Detection Workflow	18
13.	Fault Recovery Architecture	19
14.	Failover and Recovery Workflow	20
15.	Multi-Cluster Coordination Model	20
16.	Workload Orchestration Framework	20
17.	Explainable Reliability Framework	21
18.	Security and Governance Architecture	22
19.	Experimental Testbed Architecture	24
20.	Evaluation Framework	25
21.	Fault Recovery Performance	26
22.	Availability and Uptime Evaluation	27
23.	Resource Utilization and Cluster Stability	27
24.	Comparative Benchmark Analysis	28
25.	Business Continuity Framework	29
26.	Financial Analytics Resilience Architecture	30
27.	Smart Manufacturing Analytics Platform	30
28.	Business Impact Dashboard	31
29.	Challenges and Limitations Roadmap	32
30.	Future Research Roadmap	32
31.	Next-Generation Resilient Analytics Architecture	33

Fig. No.	Figure Title	Page No.
32.	Research Contribution Framework	33

List of Tables

Table. No.	Table Title	Page No.
1.	Executive summary of contributions and outcomes	7
2.	Research challenges and gaps	9
3.	Comparative analysis of existing approaches	11
4.	Functional and non-functional requirements matrix	12
5.	Data engineering components and responsibilities	15
6.	Monitoring metrics and thresholds	17
7.	Reliability assessment indicators	22
8.	Governance and compliance controls	23
9.	Experimental configuration details	24
10.	Metrics definition matrix	25
11.	Experimental results summary	26
12.	Comparative benchmark analysis	28
13.	Reliability and continuity assessment matrix	29
14.	Industry use cases matrix	31
15.	Cost-benefit analysis	31
16.	Recovery rules per mode	34

Contributions

Sl. No.	Section Topic	Reg. No. of Team Member contributed	No. of Words
	Executive Summary, Abstract and Keywords	26MML0015	490
1.	I. Introduction	26MML0047	867
2.	II. Problem Statement	26MML0006	501
3.	III. Background and Literature Review	26MML0047	1491
4.	IV. Requirements Analysis for Resilient Analytics Systems	26MML0006	547
5.	V. Multi-Cluster Spark Analytics Architecture	26MML0015	963
6.	VI. Distributed Data Engineering Layer	26MML0006	980
7.	VII. Spark Processing and Execution Layer	26MML0006	943
8.	VIII. Fault Detection and Monitoring Framework	26MML0054	1003
9.	IX. Fault-Tolerant Recovery and Resilience Layer	26MML0015	1019
10.	X. Multi-Cluster Coordination and Orchestration Layer	26MML0015	799
11.	XI. Explainability and Reliability Assessment Layer	26MML0054	846
12.	XII. Security, Governance, and Compliance Layer	26MML0047	651
13.	XIII. Experimental Setup	26MML0006	782
14.	XIV. Evaluation Metrics	26MML0054	505
15.	XV. Results and Findings	26MML0054	668
16.	XVI. Comparative Analysis	26MML0006	505
17.	XVII. Reliability and Business Continuity Assessment	26MML0047	534
18.	XVIII. Industry Use Cases and Case Study Evaluation	26MML0047	671
19.	XIX. Business Impact Assessment	26MML0054	344
20.	XX. Challenges and Limitations	26MML0015	485
21.	XXI. Future Research Directions	26MML0054	399
22.	XXII. Conclusion	26MML0015	392
	References (127)	All members	not counted
	Appendix and Data Availability	26MML0006	221

Consolidated Report on contributions		
Sl. No.	Reg. No. of Student	Total number of Words contributed across the sections
1	26MML0054	3765
2	26MML0006	4479
3	26MML0047	4214
4	26MML0015	4148
Total number of words		16606

Resilient Data Science Programming for Fault-Tolerant Distributed Analytics in Multi-Cluster Spark Environments

Dr. M Rajasekhara Babu[1], [[Student name]] (26MML0054)[2], [[Student name]] (26MML0006)[3], [[Student name]] (26MML0047)[4], [[Student name]] (26MML0015)[5]
School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore-632014, T.N., India
[1]mrajasekharababu@vit.ac.in, [2]26MML0054 [VIT e-mail], [3]26MML0006 [VIT e-mail], [4]26MML0047 [VIT e-mail], [5]26MML0015 [VIT e-mail]
Corresponding author: M Rajasekhara Babu (e-mail: mrajasekharababu@vit.ac.in).

ABSTRACT Distributed analytics platforms such as Apache Spark recover well from the loss of individual tasks, but an analytics job still fails when the cluster that runs it becomes unavailable, and long lineage chains make recovery slow even when the cluster survives. This paper presents RMC-Spark, a resilience layer that coordinates several independent Spark clusters and makes data science pipelines resumable across them. RMC-Spark combines five mechanisms: a health monitor that scores each cluster from five signals with an exponentially weighted anomaly detector; an adaptive checkpointing policy that saves a pipeline stage once the unsaved work reaches the Young and Daly optimum interval derived from an online estimate of the mean time between failures; a progress-aware failover controller that migrates only stalled jobs; a cost-based placement function; and an explainable reliability score that records the cause of every recovery decision. A small programming library turns ordinary PySpark functions into idempotent stages whose committed outputs live in shared object storage, so a job resubmitted on another cluster restores finished stages instead of recomputing them. The system is implemented with Docker, Spark 3.5, MinIO, Prometheus and a Python controller, and its decision logic is verified by 56 automated tests, including a simulated cluster outage. An evaluation methodology with nine reproducible fault types, four workloads and five compared configurations is specified together with statistical tests. Measured results are reported in Section XV once the experiment matrix has been executed. The design targets banking, healthcare, manufacturing and other domains in which analytics downtime has direct business cost.

INDEX TERMS Apache Spark, Cloud Analytics, Data Engineering, Distributed Analytics, Fault Tolerance, High Availability, Multi-Cluster Computing, Resilient Data Science

EXECUTIVE SUMMARY

Organisations increasingly depend on distributed analytics for decisions that cannot wait, from fraud screening to production-line monitoring [1][2]. Apache Spark has become the common engine for such workloads because one programming model covers batch, streaming, SQL and machine learning [3][4]. Its built-in fault tolerance, however, is scoped to a single cluster: lost partitions are recomputed from lineage and failed tasks are retried, but a failed master, a network partition or a lost availability zone stops every job that depends on that cluster [5][6].

This work treats resilience as a property of the data science program and of the platform together. On the programming side, the proposed rds library lets an analyst mark pipeline stages; each stage becomes idempotent, records its timing, and is checkpointed to shared storage according to an

adaptive policy. On the platform side, a controller observes all clusters, explains its assessment of each one, moves jobs away from a failed cluster only when they have actually stopped making progress, and resumes them from their last committed stage elsewhere. Table 1 summarises the contributions and their current status.

TABLE 1
EXECUTIVE SUMMARY OF CONTRIBUTIONS AND OUTCOMES

Contribution	What it provides	Status / outcome
C1 Health monitoring	Health score H from five weighted signals with EWMA z-score anomaly detection (Section VIII)	Implemented; thresholds and weights fixed in configuration; unit tested
C2 Adaptive checkpointing	Young/Daly interval driven by an online MTBF estimate; commit-marker protocol on shared storage (Section IX)	Implemented in the rds library; formulas unit tested
C3 Cross-cluster failover	Health state machine plus progress-aware migration; restores committed stages on the target cluster (Section IX)	Implemented; simulated outage fails over within 16 s versus at least 30 s for a timeout baseline
C4 Workload orchestration	Placement cost combining load, health and data transfer (Section X)	Implemented; deterministic tie-breaking tested
C5 Explainable reliability	Decision log with per-signal contributions and a reliability score R (Section XI)	Implemented; contributions sum to R by construction
Evaluation methodology	9 fault types, 4 workloads, 5 configurations, Mann-Whitney U, Cliff's delta, bootstrap intervals (Sections XIII-XIV)	Harness implemented; measured results pending, see Section XV
Open artefacts	Source code, configurations, datasets scripts, raw logs	Repository and data-availability statement provided

The expected industrial value lies in converting outages that currently end in failed jobs and manual reruns into automatic, explained recoveries with bounded lost work. Whether that expectation holds, and at what overhead, is decided by the experiments defined in Sections XIII and XIV; the numbers are deliberately not anticipated in this summary.

I. INTRODUCTION

A. EVOLUTION OF DISTRIBUTED ANALYTICS

Large-scale analytics moved from specialised parallel databases to commodity clusters when MapReduce showed that a simple functional model, combined with automatic partitioning and re-execution of failed tasks, could process very large datasets on unreliable hardware [7]. Hadoop generalised this idea and, with YARN, separated resource management from the processing model so that several frameworks could share one cluster [8]. The next step was in-memory processing: resilient distributed datasets (RDDs) kept intermediate data in memory and replaced data replication with lineage, a record of the coarse-grained transformations that produced each partition, so that lost partitions could be recomputed rather than restored from copies [5]. Spark later unified batch, interactive SQL, streaming and machine learning on top of this abstraction [3][9][10], while engines such as Flink pursued a streaming-first design with distributed snapshots [11]. Fig. 1 summarises this progression.

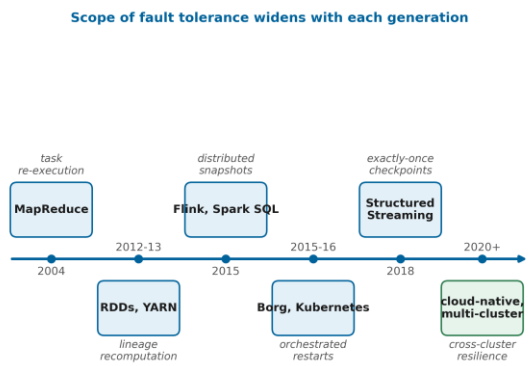


FIGURE 1. Evolution of distributed analytics platforms from batch MapReduce to cloud-native, multi-cluster analytics. Each generation widened the scope of fault tolerance, from task re-execution to lineage, to distributed snapshots, to cluster-level orchestration.

Comparative studies continue to show that Spark outperforms disk-based MapReduce on iterative and interactive workloads, which explains its dominance in data science practice [4][12]. At the same time, analytics has spread into the cloud: container orchestrators derived from Borg and Kubernetes schedule analytics jobs next to services [13][14], serverless platforms offer analytics without cluster management [15][16], and data lakes and lakehouses decouple storage from compute [17][18]. Streaming analytics has matured in parallel, with modern systems offering exactly-once processing over unbounded data [19][20][21].

B. THE RESILIENT DATA SCIENCE ECOSYSTEM

The result is an ecosystem in which a single analytical outcome depends on many independently failing parts: object storage, metadata services, cluster managers, executors, network links and the data science code itself. Fig. 2 shows the actors involved. Data engineers build ingestion and transformation pipelines, data scientists build features and models on top of them, and platform engineers operate the clusters. Each group sees failures differently. For a platform engineer, a lost worker is routine; for a data scientist whose three-hour training pipeline restarts from the beginning, the same event is a lost afternoon. Resilience therefore has to be designed into both the platform and the programs that run on it [6][22].

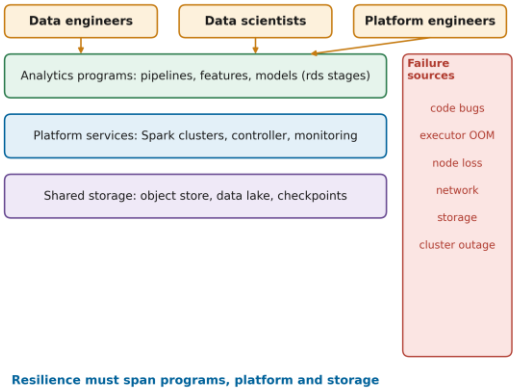


FIGURE 2. The resilient data science ecosystem: programs, platform services and shared storage, together with the people who build and operate them. Resilience must cover all layers, not only the compute engine.

C. FAULT TOLERANCE REQUIREMENTS OF MODERN ANALYTICS

Three trends make fault tolerance harder than it was when lineage was introduced. First, pipelines have become longer: feature engineering, model training and evaluation are chained into workflows whose lineage spans dozens of stages, and recomputing such lineage after a failure can take longer than the original run [23]. Second, failures are less often clean crashes. Gray failures, in which a component is degraded but still answers health checks, are a major source of availability loss in cloud systems [24]. Third, organisations increasingly run several clusters, across availability zones, clouds or business units, to obtain isolation and capacity [25][26]. These clusters are usually operated independently, so a job cannot benefit from healthy capacity in a neighbouring cluster when its own cluster fails.

D. MOTIVATION AND CONTRIBUTIONS

The motivation for this work is the gap between the fault tolerance that Spark provides inside one cluster and the availability that analytics consumers expect across clusters. Existing remedies either stay inside one cluster, for example lineage checkpointing [23] or write-ahead lineage [27], or treat analytics jobs as opaque units that are restarted from scratch by a generic high-availability script. Recent work on multi-cloud Spark scheduling with micro-checkpointing shows growing interest in the problem but relies on learned schedulers whose decisions are difficult to explain to operators [28].

The question is not only technical. Analytics teams often measure their own success by the timeliness of reports and models, while platform teams measure availability of infrastructure, and the two measures can disagree: a platform can be ninety-nine per cent available while a single outage at the wrong moment causes a business-critical pipeline to miss its deadline [29]. Resilient data science programming, as used in this paper, means writing analytics code and operating its platform so that such outages cost minutes of automatic recovery instead of hours of manual reruns.

This paper makes the following contributions. (1) It formulates resilient data science programming as the combination of resumable program stages and a cross-cluster recovery controller. (2) It designs and implements RMC-Spark, consisting of a health monitor (C1), adaptive checkpointing based on the Young and Daly optimum interval (C2), progress-aware cross-cluster failover (C3), cost-based workload placement (C4) and an explainable reliability score (C5). (3) It provides rds, a small PySpark library that makes pipeline stages idempotent and resumable on any cluster through a commit-marker protocol. (4) It defines a reproducible evaluation methodology with nine fault types, four workloads and five compared configurations, and releases the complete testbed. (5) It analyses the design with respect to requirements, security, business continuity and industrial use cases.

The rest of the paper is organised as follows. Section II states the problem and Section III reviews related work. Sections IV to XII present requirements, architecture and the individual layers. Sections XIII and XIV define the experimental setup and metrics, Sections XV to XVII report results and reliability assessment, Sections XVIII and XIX discuss industrial use and business impact, and Sections XX to XXII cover limitations, future work and conclusions.

II. PROBLEM STATEMENT

A distributed analytics job in a multi-cluster environment is exposed to failures at several levels. Fig. 3 classifies the scenarios considered in this work. At the task level, an executor process can crash because of memory exhaustion or a software defect; Spark handles this case by re-running the affected tasks [5]. At the node level, a worker machine can fail, taking its executors and cached partitions with it. Large-scale studies show that node and disk failures are frequent enough in production clouds to justify dedicated prediction models [30][31][32][33]. At the cluster level, the master, the network or an entire availability zone can become unavailable, at which point every job of that cluster stops. Finally, gray failures, such as network delay, a slow disk or a CPU-starved node, degrade performance without producing an error [24].

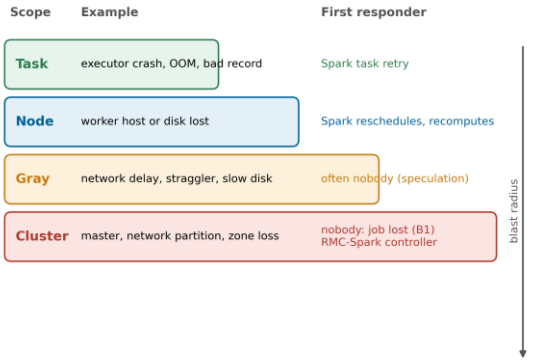


FIGURE 3. Failure scenarios considered in this work, ordered by blast radius: task, node, gray and cluster-level failures, with the component that reacts first in a standard Spark deployment.

These failures lead to five concrete problems. Node failures and cluster outages interrupt jobs; with standard Spark, an outage of the cluster that hosts a job ends the job, and someone has to resubmit it. Workload imbalance appears when stragglers or overloaded clusters slow a job down, which speculative execution only partly addresses

[34]. Data loss occurs when intermediate results existed only in executor memory or on local disks of failed nodes, forcing long lineage recomputation [23]. Performance degradation follows from both recomputation and gray failures, and operational downtime accumulates while operators diagnose and restart jobs by hand. Studies of large cloud data centres observe that hardware and software faults still cause frequent task and job failures despite high reliability targets, which motivates handling them proactively rather than by manual reruns [35].

The problem can be stated more precisely with a simple model. A job J is a sequence of stages s_1 to s_n with compute times c_1 to c_n, executed on one of K clusters. Cluster k fails as a renewal process with mean time between failures M_k; when it fails during stage s_i, all work since the last durable result is lost unless it can be recovered from lineage on surviving executors. The objectives are to maximise the probability that J completes before its deadline, to minimise the expected completion time including recovery, and to keep the overhead in failure-free executions small. Every recovery action must, in addition, be justified by observable evidence that an operator can inspect afterwards.

The research problem addressed here is therefore: how can a data science pipeline running on one of several independent Spark clusters continue after a failure of its cluster, with bounded lost work, without restarting jobs unnecessarily, and with decisions that operators can understand and audit? Table 2 relates this problem to the gaps that remain in the literature reviewed in Section III. Surveys of fault tolerance for cloud and distributed analytics identify proactive, cross-layer and explainable approaches as open directions [6][36][37], and service-level objective research argues that availability targets must be enforced dynamically rather than assumed [29][38].

TABLE 2
RESEARCH CHALLENGES AND GAPS

Challenge	State of practice	Gap addressed here
Cluster outage	Job fails; manual or scripted restart from scratch	Automatic failover with stage-level resume
Long lineage	Recompute from source or checkpoint everything	Checkpoint interval derived from observed failure rate
Gray failures	Usually invisible to health checks	Latency and progress signals in the health score
Unnecessary restarts	Timeout-based HA restarts healthy jobs	Migrate only jobs that stopped progressing
Opaque decisions	Learned or scripted policies without rationale	Per-signal contributions logged for every decision
Reproducible evaluation	Ad hoc fault experiments	Scripted fault injection and statistical comparison

III. BACKGROUND AND LITERATURE REVIEW

This section reviews the work on which RMC-Spark builds and against which it is positioned. The literature is organised by the taxonomy of Fig. 4: recovery mechanisms inside the analytics engine, checkpoint optimisation, straggler and gray-failure handling, failure prediction and AIOps, orchestration across clusters and clouds, and resilience engineering practice. Table 3 compares representative approaches.

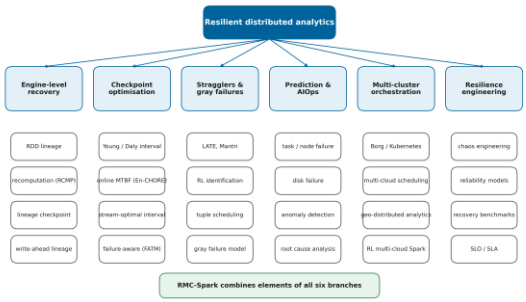


FIGURE 4. Taxonomy of the reviewed literature on resilient distributed analytics. The six branches correspond to the subsections of Section III; RMC-Spark combines elements of every branch.

A. FAULT TOLERANCE INSIDE THE SPARK ENGINE

Spark's core recovery mechanism is lineage. Each RDD records the deterministic transformations that produced it, so a lost partition is rebuilt by re-running those transformations on the surviving inputs, and checkpointing to reliable storage is offered as an option for long lineage graphs [5]. The same principle underlies Spark SQL and Structured Streaming, where query plans and write-ahead logs of offsets allow restart after failure [9][10]. Surveys of the Spark ecosystem describe lineage-based fault tolerance as one of the reasons for Spark's success, but also note the cost of recomputation for iterative and long-running applications [4].

Several studies refine this mechanism. RCMP argued, before Spark's dominance, that recomputation can replace replication of intermediate outputs as a first-order resilience strategy if task outputs are persisted selectively [39]. Kweun et al. observed that long lineage harms both performance and reliability and proposed checkpointing the lineage graph rather than the data itself [23]. Write-ahead lineage combines lineage replay with logging so that pipelined query engines can recover lost tasks without global rollback [27]. Other work targets specific Spark subsystems: partition balancing with checkpoint-based cache recovery [40], erasure-coded distributed caching with LT codes [41], and fault-tolerance optimisation for iterative graph algorithms in GraphX [42]. A recent systematic survey of fault-tolerant solutions for distributed data analytics classifies these mechanisms and identifies cross-layer, adaptive designs as an open direction [6]. All of these approaches assume that the cluster survives; none of them resumes a job on a different cluster.

B. CHECKPOINT INTERVAL OPTIMISATION

Choosing how often to checkpoint is a classical problem. Young derived a first-order approximation of the optimum checkpoint interval as the square root of twice the product of the checkpoint cost and the mean time between failures [43], and Daly extended it with higher-order terms that remain accurate when the checkpoint cost is not small relative to the MTBF [44]. Later work adapts these results to systems whose MTBF fluctuates: Sigdel et al. track failures during execution to estimate the MTBF in real time and skip checkpoints that would

not pay off [45]. In stream processing, Jayasekara et al. derived a utilisation-optimal checkpoint interval and validated it in a cloud setting [46][47]; similar analyses exist for Flink applications [48], and FATM triggers checkpoints in line with the observed failure rate instead of periodically [49]. Incremental checkpointing reduces the cost of each checkpoint [50], and convergence-aware policies have been proposed for deep learning training jobs [51]. RMC-Spark applies the Young and Daly interval at the granularity of data science pipeline stages and couples it with an MTBF estimate maintained by the cross-cluster controller.

C. FAULT RECOVERY BENCHMARKS FOR ANALYTICS FRAMEWORKS

Empirical comparisons of recovery behaviour are the closest benchmarks to this work. Van Dongen and Van den Poel tested Spark Streaming, Structured Streaming, Flink and Kafka Streams under master, driver, worker, application and task failures, and reported outage, downtime, recovery time, data loss and duplicate processing [52]. Vogel et al. repeated such an analysis in a cloud-native Kubernetes environment with chaos-engineering-style fault injection and found that Flink was the most stable while Structured Streaming recovered reliably at a higher latency [53]. PGVal validates processing guarantees end to end and introduces reliability and failure-cost metrics for stream processors [54]. A survey of stream processing systems traces how fault-tolerance mechanisms evolved from replication to distributed snapshots [19]. These studies benchmark single clusters; this paper reuses their metric definitions, notably the throughput-based recovery criterion, and extends the scenario space to whole-cluster outages.

D. STRAGGLERS AND GRAY FAILURES

Stragglers have been studied since the first MapReduce deployments. LATE improved speculative execution for heterogeneous environments [55], and Mantri identified causes of outliers and acted on them early [56]. More recent approaches learn to identify stragglers, for example with reinforcement learning on heterogeneous Spark clusters [57] or neural networks over task features [58], and survey work classifies straggler mitigation for Hadoop [34]. In streaming systems, Hone schedules outgoing tuples to minimise backlog and avoid stragglers [59]. Gray failures are broader than stragglers: Huang et al. describe them as differential observability, where a component looks healthy to one observer and failed to another [24]. RMC-Spark addresses gray failures by combining master reachability with probe latency and job progress observed from the driver.

E. FAILURE PREDICTION AND AIOPS

A large body of work predicts failures before they happen. Bi-LSTM models predict task and job failures from data centre traces [35], comparative studies evaluate traditional and deep learning predictors on Google cluster traces [60], and failure analyses of machine learning jobs in the cloud characterise when jobs fail [61]. Proactive schedulers combine such predictors with remedial actions [62], and online VM failure prediction has been integrated with tolerance mechanisms for high availability [63]. Node failure prediction has been deployed on very large platforms [30][31], disk failure prediction has been adapted to minority disk models and to streaming settings [32][33], and workload prediction models support capacity decisions [64].

AIOps generalises these ideas to IT operations. Notaro et al. categorised failure management methods by their time of intervention [65]; later surveys connected AIOps with MLOps [66] and with large language models [67]. Anomaly detection over metrics and logs is surveyed extensively [68][69][70][71], including a mapping study of monitoring data used for cloud anomaly detection [72] and anomaly detection for Kubernetes infrastructures [73]. Root cause analysis for microservices ranges from graph-based ranking [74] to explanations of cascading failures from logs [75], and surveys summarise both detection and localisation [76][77]. A recurring finding is that operators need interpretable outputs; Li et al. stress that AIOps solutions must be trustable and interpretable to be adopted [30]. RMC-Spark therefore uses a transparent additive health model rather than an opaque predictor.

F. ORCHESTRATION ACROSS CLUSTERS AND CLOUDS

Cluster managers such as Borg introduced large-scale scheduling with fault-tolerant control planes [13], and the lessons from Borg and Omega shaped Kubernetes [14]. Kubernetes scheduling has been surveyed from several angles [78][79][80], cost-efficient and simulation-driven container orchestration strategies have been proposed [81][82], and container orchestration has been brought to HPC systems [83]. For multiple clouds, systematic reviews describe orchestration frameworks [25], and scheduling methods account for security and reliability constraints [84]. Geo-distributed analytics research optimises data placement and network transfers across data centres [85][86][87][88][89], as summarised by a survey of bandwidth-aware frameworks [90]. Raj et al. recently combined a reinforcement-learning scheduler with predictive provisioning and fault-tolerant micro-checkpointing for Spark workflows in multi-cloud systems [28]; RMC-Spark shares the multi-cluster goal but uses an analytical checkpoint policy and an explainable placement cost, and it is evaluated with real fault injection.

G. CLOUD FAULT TOLERANCE AND RESILIENCE ENGINEERING

Reliability research for clouds covers proactive fault tolerance in federations [91], failure-risk-based proactive actions [92], service-oriented reliability modelling [93] and analytical availability models for edge, fog and cloud [94]. Surveys catalogue fault tolerance methods for cloud and distributed computing [22][36][37]. Chaos engineering, introduced at Netflix as experimentation on production systems [95], has since been applied to cloud security [96], containerised applications [97] and digital twins [98], and a recent multi-vocal

review consolidates its practices [99]. Finally, configuration tuning affects both performance and stability of Spark: auto-tuning surveys and systems [100][101][102][103], performance prediction [104], interference diagnosis [105] and engine comparisons [106] show how sensitive analytics performance is to its environment.

H. DATA PLATFORMS AND PIPELINE RELIABILITY

Resilience also depends on how data are stored and governed. Data lake research has examined architectures and metadata management for raw and derived data [17][107], and lakehouse designs add transactional guarantees on top of object storage [18][108]. These guarantees matter for recovery, because a resumed job must be able to tell committed results from partial ones. Studies of data pipeline quality find that many pipeline failures originate in data problems rather than infrastructure faults [109], and holistic quality frameworks propose continuous checks along the pipeline [110]. Schema evolution adds a further risk when stored intermediate results outlive the code that produced them [111]. RMC-Spark adopts object storage as its durable layer and uses a lightweight commit protocol rather than a full table format, which keeps the dependency footprint small while preventing partially written stages from being read.

I. SYNTHESIS

Three observations emerge from the review. First, mechanisms for reducing recovery cost inside one engine are mature, and optimal checkpointing theory is well established, but the two are rarely combined with information about the actual failure rate of the platform. Second, failure prediction and anomaly detection are active fields, yet their outputs seldom drive concrete recovery actions for analytics jobs, and when they do, the decision logic is often opaque. Third, multi-cluster and multi-cloud work focuses on placement and scheduling efficiency more than on continuing a running job after its cluster disappears. These observations define the space in which RMC-Spark is positioned.

J. COMPARATIVE ANALYSIS AND POSITIONING

Table 3 compares representative approaches along five properties required by Section II: recovery from whole-cluster loss, adaptive checkpointing, gray-failure awareness, explainable decisions and reproducible fault-injection evaluation. No reviewed approach covers all five. Engine-level methods are efficient but bound to one cluster; prediction and AIOps methods detect problems but rarely act on analytics jobs; multi-cloud schedulers act across clusters but explain little. RMC-Spark is designed to cover the combination, at the price of the overheads and single-host limitations discussed in Section XX.

TABLE 3
COMPARATIVE ANALYSIS OF EXISTING APPROACHES

Approach	Main idea	Cross-cluster recovery	Adaptive checkpoint	Gray-failure aware	Explainable	Fault injection	Analytics engine
RDD lineage [5]	Recompute lost partitions	No	No	No	n/a	No	Spark
RCMP [39]	Recomputation instead of replication	No	Partial	No	n/a	Yes	Hadoop
Lineage checkpoint [23]	Checkpoint lineage graph	No	No	No	n/a	Yes	Spark
Write-ahead lineage [27]	Log lineage at runtime	No	No	No	n/a	Yes	Quokka
En-CHORE [45]	Online MTBF, skip checkpoints	No	Yes	No	n/a	Simulated	Generic
Optimal stream checkpoint [47]	Utilisation-optimal interval	No	Yes	No	n/a	Yes	Flink
FATM [49]	Failure-rate-driven checkpoints	No	Yes	No	n/a	Yes	Stream
Hawkeye [57]	RL straggler identification	No	No	Yes	Low	No	Spark
Bi-LSTM prediction [35]	Predict task/job failures	No	No	Partial	Low	Trace	Cloud
Proactive scheduling [62]	Predict and remediate	No	No	Partial	Low	Trace	Cloud
Recovery benchmark [53]	Chaos-based comparison	No	No	No	n/a	Yes	Flink, Spark, Kafka
Multi-cloud RL scheduler [28]	RL scheduling, micro-checkpoints	Yes	Yes	Partial	Low	Not stated	Spark
RMC-Spark (this work)	Health-driven failover, resumable stages	Yes	Yes	Yes	Yes	Yes	Spark

IV. REQUIREMENTS ANALYSIS FOR RESILIENT ANALYTICS SYSTEMS

Requirements were derived from the problem statement, from the gaps in Table 2, and from service-level practice in cloud computing [29][38]. Fig. 5 shows the requirements engineering framework: stakeholder concerns are refined into functional and non-functional requirements, each requirement is traced to a component, and each component is traced to a test or an experiment.

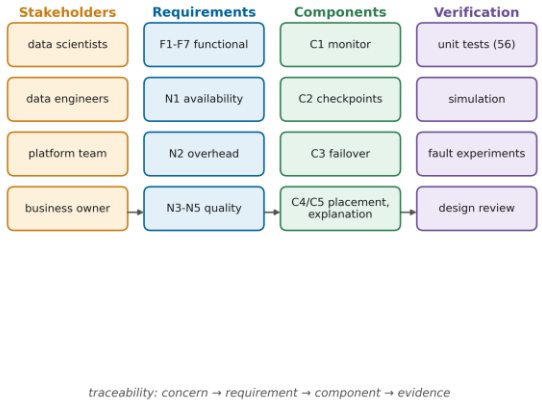


FIGURE 5. Requirements engineering framework. Stakeholder concerns are refined into requirements that are traced to components (C1-C5) and verified by unit tests or by the experiments of Section XIII.

Scalability requires that adding a cluster or a workload does not change the controller's design; the cluster registry is therefore data driven and the control loop is linear in the number of clusters and jobs. Availability requires that an analytics job survives the loss of the cluster that runs it, provided another cluster is healthy. Recoverability requires bounded lost work, which translates into the checkpoint policy of Section IX. Reliability requires that recovery does not itself cause harm, for example by restarting jobs that were still progressing. Interoperability requires that existing PySpark code can adopt the approach with small changes, which motivates a decorator-based library instead of a new language. Governance requires that every automatic decision is recorded with its reason and that data movement follows organisational policy [112][113]. Operational continuity requires that the system degrades gracefully: if no healthy cluster exists, jobs wait rather than fail. Data quality concerns, frequently identified as root causes of pipeline problems [109][110], require that half-written intermediate outputs are never consumed.

A. SCALABILITY AND AVAILABILITY REQUIREMENTS

Scalability is considered along three axes: the number of clusters, the number of concurrent jobs and the size of the data. The controller's work per poll grows linearly with clusters and running jobs, each poll costs one HTTP request per cluster and two per running driver, and data volume affects only the cost of checkpoints, which the adaptive policy accounts for. Availability is expressed as a job-level property: a job submitted while at least one cluster is healthy should complete, even if the cluster it started on fails. This is stronger than infrastructure availability, because it requires the job's progress to survive, not only the platform.

B. RECOVERABILITY AND RELIABILITY REQUIREMENTS

Recoverability is quantified by two bounds. The time bound requires that a cluster failure is detected and acted upon within a few polling intervals plus a small grace period for progress. The work bound requires that the work redone after a failure is limited by the checkpoint interval rather than by the total job length. Reliability requires that recovery decisions are correct in the sense of not making things worse: a job that would have finished on its own must not be restarted, a single slow probe must not cause a failover, and a recovered cluster must not oscillate between states.

C. INTEROPERABILITY, GOVERNANCE AND CONTINUITY REQUIREMENTS

Interoperability requires that the approach works with standard Spark releases, standard storage interfaces and standard PySpark code, so that adoption does not require a platform migration. Governance requires that automatic actions are attributable and auditable and that policies about data location can be expressed. Operational continuity requires graceful degradation: when no cluster is available, the system waits and resumes rather than failing, and when the controller itself restarts, committed work remains usable because it is stored outside the controller. Table 4 lists the resulting requirements and traces each to its component and verification method.

TABLE 4
FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS MATRIX

ID	Requirement	Component	Verification
F1	Probe every cluster and running driver periodically and compute a health score	C1 monitor, health model	Unit tests; Fig. 23 time series
F2	Detect a failed cluster within a few polling intervals without reacting to single glitches	C1 state machine	Unit tests (consecutive-poll rule)
F3	Checkpoint pipeline stages adaptively and commit them atomically	C2 rds library	Unit tests; checkpoint metrics
F4	Resume a job on another cluster from its last committed stage	C2 + C3	Cluster-outage experiments
F5	Migrate only jobs that stopped progressing	C3 jobs manager	Simulation test; master-kill experiments
F6	Place jobs on the best available cluster	C4 orchestrator	Unit tests
F7	Record every decision with its cause and expose reliability scores	C5 decision log	Unit tests; API
N1	Availability: no job loss when one cluster fails and another is healthy	Whole system	Success rate under cluster faults
N2	Overhead: limited slowdown in fault-free runs	C2 policy	Clean-run overhead, Eq. (11)
N3	Reproducibility: every experiment scripted and seeded	Harness	Repository, configuration files
N4	Interoperability: unchanged Spark, standard PySpark APIs	rds library	Workloads use stock Spark 3.5
N5	Security and governance: auditable actions, least privilege in production	Section XII	Design review

V. MULTI-CLUSTER SPARK ANALYTICS ARCHITECTURE

A. DESIGN PRINCIPLES

RMC-Spark is built on four design principles. The first is to keep the analytics engine unmodified. Every cluster runs stock Apache Spark in standalone mode, so the approach applies to existing deployments and benefits from upstream improvements. The second is to separate jobs from clusters: a job is identified by a stable job identifier, its durable state lives in shared object storage, and the cluster on which an attempt runs is an interchangeable resource. The third is to act on evidence of harm rather than on suspicion. A cluster that looks unhealthy is avoided for new work, but a running job is moved only when it has stopped making progress. The fourth is to explain every action, because automatic recovery that operators cannot audit is unlikely to be trusted in production [30][65].

B. OVERALL ARCHITECTURE

Fig. 6 shows the architecture as deployed in the testbed. Two Spark clusters, A and B, each consist of one master and two workers. A shared MinIO object store, accessed through the Hadoop S3A connector, holds input data, stage checkpoints, streaming checkpoints and results. Prometheus scrapes metrics from the Spark masters, the workers and the controller, and Grafana visualises them. The resilience controller runs in its own container and hosts the Spark drivers: every job is launched with spark-submit in client deploy mode from the controller, so the driver process lives outside the cluster whose executors it uses. A fault injector controls the containers of each cluster through the Docker Engine API.

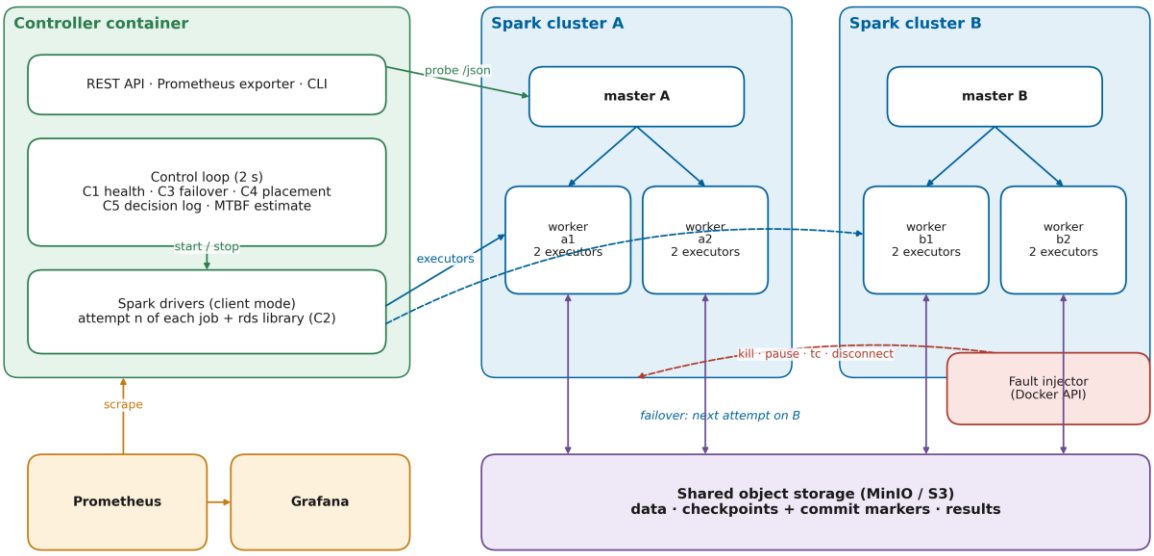


FIGURE 6. Resilient multi-cluster Spark architecture for fault-tolerant distributed analytics (core figure). Drivers run in the controller, executors run on cluster A or B, and all durable state lives in shared object storage, so a job can resume on either cluster.

Placing drivers in the controller is the key structural decision. In Spark's cluster deploy mode, the driver runs on a worker of the same cluster and is lost together with it; in client mode, the driver survives the loss of the cluster and can be stopped deliberately and restarted against another master. The controller can then observe both sides of a failure: the master's view of the cluster and the driver's view of task progress. This dual observation is what allows the system to distinguish a dead master with healthy executors from a dead cluster, a distinction that matters for gray failures [24].

C. COMPONENTS AND INTERACTIONS

The controller executes a control loop every two seconds. In each iteration it probes the JSON endpoint of every Spark master, queries the REST API of every running driver, computes a health score per cluster (C1), updates a failure detector state machine per cluster, applies the recovery rules to every job (C3), and, when a job must start or move, selects a cluster with the placement cost (C4). Every state transition and recovery action is appended to a decision log with the contributions of each health signal (C5). The data science programs use the rds library, which implements resumable stages and the adaptive checkpoint policy (C2). The policy needs an estimate of the mean time between failures, which the controller maintains from the observed cluster failures and passes to each job attempt as an argument.

The interaction during a cluster outage proceeds as follows. When cluster A fails, the probes of its master time out and its health score drops; after three consecutive low scores the state machine declares A failed. The driver of a job running on A reports no completed tasks any more, so after a short stall threshold the controller terminates the driver, selects cluster B, and launches a new attempt with the same job identifier against B's master. The new attempt finds the commit markers of stages finished by the first attempt in object storage, restores those stages, and computes only the remaining ones.

D. LAYERED ANALYTICS STACK

Fig. 7 presents the same system as a layered stack, which also structures Sections VI to XII. The storage layer provides durable, shared object storage. The processing layer contains the Spark clusters. The resilience programming layer consists of the rds library inside each driver. The control layer contains monitoring, detection, recovery and orchestration. The explainability layer turns control decisions into auditable records and reliability scores, and the governance layer spans all of them with security controls and policies. Monitoring and observability cut across the stack, following current practice for container-based systems [114][115].

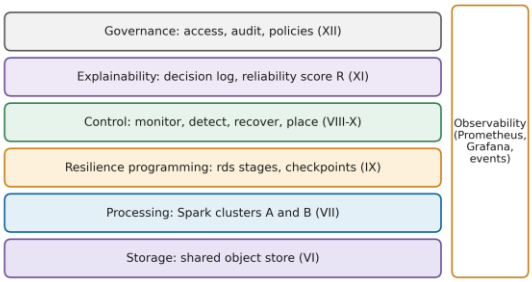


FIGURE 7. Layered analytics stack of RMC-Spark. Observability and governance span all layers; each layer is detailed in one of Sections VI to XII.

E. FAILURE DOMAINS AND ASSUMPTIONS

The architecture distinguishes three failure domains. A cluster domain contains a master and its workers; the design tolerates the loss of any number of cluster domains as long as one remains. The storage domain contains the object store; the design assumes it is available, which in production means a replicated service. The control domain contains the controller and the drivers; the design assumes that the controller is available while it is making decisions, and in production this is achieved with replicas. Faults are assumed to be fail-stop or fail-slow; Byzantine behaviour, such as a cluster returning wrong results, is out of scope. Stage functions are assumed to be deterministic, which is the same assumption that lineage-based recovery in Spark relies on [5].

F. DEPLOYMENT MODEL

The testbed deploys all components with Docker Compose on a single host, which makes the experiments reproducible and lets faults be injected precisely. The architecture does not depend on this choice. In production, clusters A and B would be separate Spark deployments in different availability zones or clouds, the object store would be a replicated cloud object store, and the controller would run with redundant replicas. Multi-cloud orchestration frameworks and geo-distributed analytics systems already address data placement and transfer across such deployments [25][90]; the placement cost of Section X includes a data-transfer term for this reason, although it is zero in the testbed where all data are shared.

VI. DISTRIBUTED DATA ENGINEERING LAYER

A. INGESTION AND STORAGE ORGANISATION

The data engineering layer stores all durable data in an S3-compatible object store organised into three buckets: data for inputs, checkpoints for stage and streaming checkpoints, and results for job outputs. Object storage decouples data from compute in the same way as data lakes and lakehouses do in production platforms [17][18][107][108], and it is the precondition for cross-cluster recovery: any cluster can read any committed stage. Fig. 8 shows the data engineering pipeline used by the workloads.

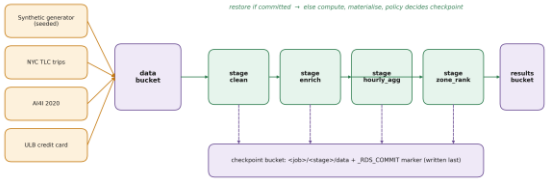


FIGURE 8. Distributed data engineering pipeline. Raw data are ingested into the data bucket, transformed by resilient stages whose committed outputs are written to the checkpoint bucket, and published to the results bucket.

Ingestion has two paths. A deterministic generator produces a synthetic taxi-trip dataset of four million records, including about one per cent of deliberately invalid rows for the cleaning stage, and a skewed directed graph of two million edges for the iterative workload. The generator is seeded, so every team member and reviewer obtains identical data. Public datasets can be added through a download script and an ingestion job: monthly NYC taxi trip records, the AI4I 2020 predictive maintenance dataset and the ULB credit-card fraud dataset. The normalisation function maps the real taxi schema onto the synthetic one, so the same pipeline runs on both.

B. TRANSFORMATION WITH RESILIENT STAGES

Transformations are written as ordinary PySpark functions decorated as stages of a ResilientPipeline. The batch ETL workload, for example, consists of a cleaning stage that removes trips with implausible duration, distance, fare or speed; an enrichment stage that joins pick-up and drop-off zones with a borough lookup; an hourly aggregation stage that computes trip counts, average and 95th-percentile fares, speeds and tip rates; and a ranking stage that selects the top revenue zones per borough with a window function. Each stage is a pure function of its inputs, which is the property that makes re-execution and restoration equivalent.

Every stage follows the same protocol. If the stage has a commit marker from an earlier attempt, its output is read back from the checkpoint bucket and the function is not executed. Otherwise the function runs, its result is persisted in memory and disk and materialised with a count so that the measured compute time is real, and the checkpoint policy decides whether to write the result to object storage. When a checkpoint is written, the data are written first and a small JSON commit marker second. Because a stage counts as committed only when the marker exists, output left half-written by a crashed attempt is never read, and a re-run simply overwrites it. Reading the committed data back also truncates

the lineage, so downstream failures no longer require recomputing upstream stages.

C. STREAMING DATA PATH

The streaming workload follows a different path because its unit of progress is a micro-batch, not a stage. Structured Streaming already records offsets and operator state in a checkpoint location after each micro-batch [10]; RMC-Spark places this location in the checkpoint bucket, so it is visible to every cluster. When a streaming job is resubmitted on another cluster with the same job identifier, the query restarts from the last committed batch and continues to write to the same output location. The job's driver writes one event per micro-batch with the number of input rows, the processing rate and the batch duration, which the analysis uses to measure throughput recovery.

D. METADATA MANAGEMENT AND SCHEMA EVOLUTION

Each commit marker records the stage name, its sequence number in the pipeline, the job identifier, the attempt number and a timestamp. These markers are the metadata catalogue of the resilience layer: from them the library reconstructs, at the start of every attempt, which stages are already committed. For strictly linear pipelines, such as the iterations of an iterative algorithm, the library uses the sequence numbers to skip all stages before the latest committed one, instead of recomputing stages whose results are already implied by a later checkpoint. Stage events with timings are written to a per-job event log, which the analysis uses to compute recomputed work and checkpoint cost.

Schema evolution is handled conservatively. Stage outputs are stored in Parquet, which embeds its schema, so a restored stage carries the schema with which it was written. Because the stage identity includes the job identifier, a change of pipeline code is expected to use a new job identifier, which prevents an old checkpoint from being combined with new code. General approaches to schema evolution in NoSQL and relational systems [111] and to automated data governance [113] could extend this policy, for example by recording a code version in the marker.

E. PARTITIONING AND STORAGE MANAGEMENT

Partitioning follows Spark's defaults with an explicit shuffle partition count of 16 and adaptive query execution enabled, which coalesces small partitions at run time. Input data are generated in 16 partitions, matching the four executor cores available per job and leaving room for load balancing. Checkpoint data are retained per job identifier, so storage grows with the number of jobs and can be reclaimed by a retention policy once a job has succeeded. Table 5 summarises the components and their responsibilities. Data pipeline studies report that unreliable intermediate data and missing validation are common root causes of pipeline failures [109][110]; the commit protocol addresses the reliability part of that problem, while data validation remains the responsibility of the pipeline code.

TABLE 5
DATA ENGINEERING COMPONENTS AND RESPONSIBILITIES

Component	Responsibility
Data bucket	Immutable inputs (synthetic and public datasets)
Checkpoint bucket	Committed stage outputs, commit markers, streaming offsets and state
Results bucket	Final outputs and JSON job summaries
Data generator	Seeded synthetic trips and skewed graph
Ingestion job	Copies downloaded public datasets into the data bucket
ResilientPipeline	Stage restore, materialisation, policy decision, atomic commit
Commit marker	Stage metadata: name, sequence, job, attempt, time
Event log	Per-stage timings and checkpoint decisions for analysis

F. DATA QUALITY IN RESILIENT PIPELINES

Resumption interacts with data quality in two ways. Positively, it prevents a class of quality incidents in which a rerun after a failure reads incomplete intermediate files and silently produces wrong aggregates; the commit protocol rules this out. Negatively, it makes stale intermediate results possible if the input data change between attempts, because a restored stage reflects the inputs that existed when it was computed. RMC-Spark treats inputs as immutable for the duration of a job, which matches the batch workloads of this study; pipelines with mutable inputs would need to include an input version in the job identifier or in the commit marker, in line with the governance practices discussed in Section XII.

VII. SPARK PROCESSING AND EXECUTION LAYER

A. SPARK EXECUTION MODEL

Each cluster runs Spark 3.5.3 in standalone mode. A standalone cluster consists of a master, which tracks workers and allocates resources to applications, and workers, which launch executor processes on behalf of applications. Fig. 9 shows the execution architecture. A Spark application starts with a driver, which builds the logical plan of the program, optimises it, splits it into jobs and stages, and schedules tasks on executors [3][9]. Executors run tasks, cache partitions and report status and heartbeats to the driver.

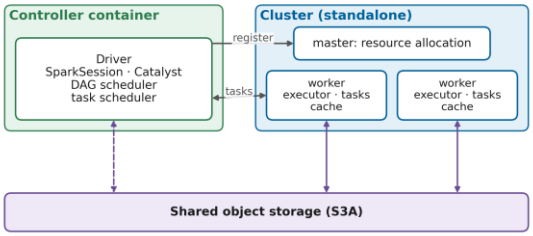


FIGURE 9. Spark execution architecture in RMC-Spark. The driver runs in the controller container in client mode; the master allocates executors on the workers of the selected cluster; executors read from and write to shared object storage.

In RMC-Spark, the controller launches each driver with spark-submit in client mode, a driver memory of 512 MB, executors with one core and 576 MB of memory, and at most four executor cores per job. The driver advertises the controller's host name, so executors on either cluster can reach it, and its user interface port is assigned by the controller so

that progress can be queried through Spark's REST API.

B. RDD LINEAGE AND DAG SCHEDULING

Spark represents a computation as a directed acyclic graph of RDDs connected by narrow and wide dependencies [5]. Narrow dependencies, such as a filter or a map, allow a partition to be computed from a single parent partition and are pipelined within one stage. Wide dependencies, such as a group-by or a join with shuffle, require data from many parent partitions and form stage boundaries. The DAG scheduler submits stages in topological order, and the task scheduler assigns the tasks of each stage to executors, preferring locations that hold cached input. Fig. 10 shows the DAG of the ETL workload together with the stage boundaries introduced by the resilient pipeline.

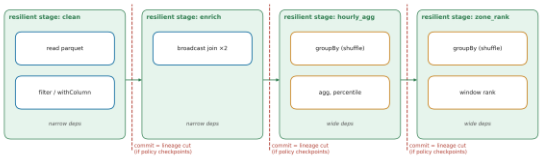


FIGURE 10. DAG processing workflow of the ETL workload. Spark stages (shuffle boundaries) are shown inside the resilient pipeline stages; a committed pipeline stage cuts the lineage, so later failures never recompute earlier stages.

Lineage determines the cost of recovery. When an executor is lost, the driver identifies the lost partitions and re-runs the minimal set of tasks needed to rebuild them, possibly going back through several shuffles if shuffle files were also lost. For pipelines with many wide dependencies this recomputation can be long, which is the long-lineage problem [23]. A committed resilient stage cuts the lineage at its boundary, because its consumers read from object storage instead of from the transformations that produced it. The checkpoint policy of Section IX decides where these cuts are worthwhile.

C. TASK EXECUTION AND BUILT-IN FAULT TOLERANCE

Spark's built-in mechanisms remain active in all configurations and form the first line of defence. A failed task is retried up to four times before its stage fails. Executors send heartbeats every 10 s, and an executor that is silent for the 60 s network timeout is declared lost, after which its tasks are rescheduled. Adaptive query execution adjusts the number of shuffle partitions at run time. In the proposed configuration, speculative execution is additionally enabled, so slow tasks are duplicated on other executors; this is the standard Spark answer to stragglers [55]. Performance prediction and configuration tuning for Spark are active research areas [100][104], but RMC-Spark deliberately uses fixed, documented settings so that differences between configurations can be attributed to the resilience mechanisms rather than to tuning.

D. RESOURCE ALLOCATION AND UTILISATION

Each worker offers two cores and 1280 MB of executor memory, so a cluster offers four cores and fits two executors per worker. The controller requests four executor cores per job, which equals one cluster's capacity and reflects a dedicated-cluster setting where one important job occupies its cluster. Resource utilisation is measured from the masters' reports of used and total cores, sampled at every control-loop iteration:

$$U = \frac{\sum_t c_{\text{used}}(t)\Delta t}{\sum_t c_{\text{total}}(t)\Delta t} \qquad (1)$$

where $c_{used}(t)$ and $c_{total}(t)$ are the cores used and offered by the cluster at sample t, and Δt is the time to the next sample. Utilisation computed with Eq. (1) over a run shows both how fully a job uses its cluster and how much capacity is idle while a job waits for recovery. Surveys of the Spark ecosystem identify resource management and scheduling as central to Spark performance [4], and Fig. 23 in Section XV uses this measure to show the shift of load from the failed to the healthy cluster.

E. INTERACTION OF BUILT-IN AND EXTERNAL RECOVERY

Built-in and external recovery must not interfere. Two interactions were considered in the design. First, Spark may still be retrying tasks when the controller decides to migrate a job; because migration happens only after ten seconds without any completed task on a cluster that the controller already considers failed, Spark's own recovery has had the opportunity to make progress and has not. Second, the controller's stall signal could be triggered by a long but healthy task; this is why stall alone can make a cluster only SUSPECT, and why migration requires the cluster to be FAILED as well. The two mechanisms are therefore layered: Spark handles what it can inside the cluster, and the controller acts only when the evidence indicates that the cluster as a whole cannot continue.

F. DISTRIBUTED PROCESSING FLOW

Combining the elements above, a job in RMC-Spark follows a fixed flow. The controller places the job and launches attempt zero. The driver registers with the master, receives executors, and the rds library restores committed stages and computes the others, checkpointing according to the policy. The driver writes stage events, and the controller reads task progress from the driver. If the job finishes, the controller records success. If the driver exits with an error or the cluster fails while the job is stalled, the controller launches the next attempt, on the same or another cluster according to the rules of Section IX, and the flow repeats from the restore step.

VIII. FAULT DETECTION AND MONITORING FRAMEWORK

A. MONITORING AND OBSERVABILITY ARCHITECTURE

Observability in RMC-Spark has two purposes: operators need dashboards to understand the platform, and the controller needs signals to make decisions. Fig. 11 shows how both are served. Spark masters and workers expose metrics in Prometheus format through Spark's metrics servlet, Prometheus scrapes them together with the controller's own metrics every five seconds, and Grafana renders a dashboard with health scores, cluster states, alive

workers, utilisation, failovers and reliability scores. Industrial surveys show that such metric pipelines, often combined with distributed tracing, are now standard for container-based systems [114][115]. The controller does not depend on Prometheus for its decisions: it probes the masters and drivers directly in its control loop, which keeps the detection path short and free of scrape delays.

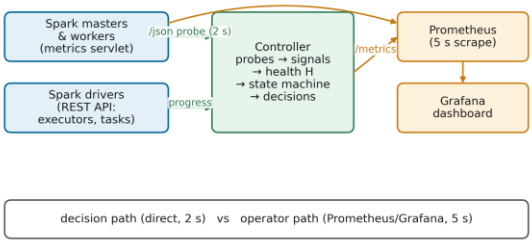


FIGURE 11. Monitoring and observability architecture. The controller probes masters and drivers directly for decisions; Prometheus and Grafana provide the operator view of the same signals.

B. HEALTH SIGNALS

The health model uses five signals per cluster, chosen to cover the failure scenarios of Fig. 3. The unreachable signal is set when the master's JSON endpoint does not answer within the 1.5 s probe timeout or reports a status other than alive. The workers-lost signal is the fraction of expected workers that the master does not report as alive. The latency signal captures gray failures: the response time of the master probe is tracked with an exponentially weighted moving average and variance, and a standardised deviation is computed as

$$z_t = \frac{x_t - \mu_{t-1}}{\sigma_{t-1}} \qquad (2)$$

where x_t is the probe latency at poll t and μ_{t-1} and σ_{t-1} are the EWMA mean and standard deviation before the poll. The anomaly level of the latency signal rises linearly from zero at $|z| = 3$ to one at $|z| = 8$. Samples whose $|z|$ exceeds the lower threshold are not absorbed into the baseline, so a long degradation cannot redefine what normal looks like, a known pitfall of adaptive detectors [68][69]. The stall signal is the time since the running job on the cluster last completed a task, divided by a 120 s limit, and the executor-loss signal is the fraction of the job's executors that the driver reports as lost. The last two signals come from the driver and therefore reflect what the job actually experiences, which is what the master alone cannot see in a gray failure [24].

C. HEALTH ASSESSMENT

Each signal i yields an anomaly level $a_{i,k}$ between zero and one for cluster k. The health score combines them additively:

$$H_k = 1 - \sum_{i=1}^{5} w_i a_{i,k} \qquad (3)$$

with weights w_i that sum to one. The weights, given in Table 6, reflect severity: an unreachable master (0.45) and lost workers (0.25) indicate loss of capacity, a stalled job (0.15) indicates loss of progress, abnormal latency (0.10) indicates degradation, and executor loss (0.05) is partly handled by Spark itself. One rule overrides the sum: a reachable master that reports zero workers has no capacity, so its health is set to zero. Because the model is additive, each signal's contribution $w_i a_{i,k}$ is directly interpretable, and the two largest contributions are logged as the cause of every state change. Additive and threshold-based detectors are less expressive than deep anomaly detectors, but their behaviour is predictable and explainable, which surveys of AIOps adoption identify as a practical requirement [65][72].

TABLE 6
MONITORING METRICS AND THRESHOLDS

Signal / parameter	Source and definition	Value	Effect
Poll interval	Controller control loop	2 s	Detection granularity
Probe timeout	HTTP request to master /json	1.5 s	Unreachable if exceeded
unreachable weight	Master probe failed or status not ALIVE	0.45	Anomaly 1 when unreachable
workers_lost weight	1 − alive / expected workers	0.25	Linear in fraction lost; 0 workers forces H = 0
latency weight	EWMA z-score of probe latency (α = 0.2)	0.10	Anomaly ramps from $z = 3$ to $z = 8$
stall weight	Time since last completed task / 120 s	0.15	Anomaly 1 after 120 s without progress
executor_loss weight	Lost / seen executors of running job	0.05	Anomaly 1 if all executors lost
Suspect threshold	H below this value	0.95	Cluster avoided for new placements
Failed threshold	H below this value for 3 consecutive polls	0.50	Cluster declared FAILED
Recovery rule	H at least 0.95 for 5 consecutive polls	—	FAILED to HEALTHY
Migration stall	No task progress on a FAILED cluster	10 s	Job migrated

Signal / parameter	Source and definition	Value	Effect
Baseline timeout (B3)	Master unreachable continuously	30 s	Naive failover

D. FAULT DETECTION WORKFLOW

Fig. 12 shows the detection workflow. The scores feed a state machine with three states per cluster. A cluster is HEALTHY while its score is at least 0.95. Any lower score makes it SUSPECT, which removes it from consideration for new work but does not affect running jobs. A cluster becomes FAILED only after three consecutive polls below 0.50, which, with a 2 s poll interval, means about six seconds of strong evidence; a single slow probe cannot trigger failover. A FAILED cluster returns to HEALTHY only after five consecutive polls at or above 0.95. This hysteresis prevents oscillation when a cluster flaps during recovery.

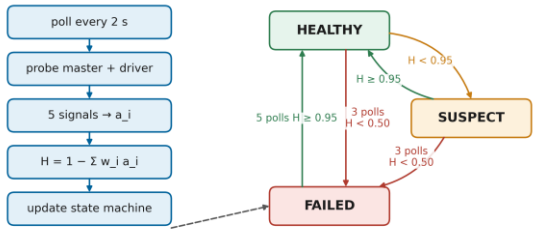


FIGURE 12. Fault detection workflow and cluster state machine. Thresholds create hysteresis: three polls below 0.50 to fail, five polls at or above 0.95 to recover.

The thresholds are set so that different failures produce different states. An unreachable master contributes 0.45 and, by construction, also counts all workers as lost, giving a score of 0.30 and hence FAILED after three polls. A fully anomalous latency signal alone yields 0.90 and a long stall alone 0.85; both make the cluster SUSPECT but not FAILED, because degradation does not justify moving a running job. The baseline B3 replaces this machine with a fixed rule, unreachable for 30 s, which represents typical scripted high-availability checks.

E. CALIBRATION AND FALSE ALARMS

Two kinds of error matter for a detector that triggers recovery. A false alarm moves or restarts a job unnecessarily; a missed detection leaves a job stuck until its deadline. The design reduces false alarms through three mechanisms: consecutive-poll confirmation, the separation of SUSPECT from FAILED, and the requirement that a job be stalled before it is migrated. Missed detections are limited by the fact that a dead master or a fully disconnected cluster always produces the unreachable signal, whose weight, together with the implied loss of workers, drives the score below the failure threshold. The latency signal needs a warm-up of five probes before it reports anomalies, so faults injected in the first ten seconds after the controller starts are detected only through the other signals; in the experiments, the controller runs continuously, so this case does not arise.

F. WORKLOAD DIAGNOSTICS AND EARLY FAILURE DETECTION

The driver-side signals provide diagnostics beyond cluster state. Because the stall and executor-loss signals are computed per job, the controller can tell whether a cluster problem actually affects the job that runs on it. This enables the progress-aware recovery rule of Section IX. The same signals are exported to Prometheus, so an operator sees the health score drop and the cause listed in the decision log at the same time. Early detection in the stronger sense of predicting failures before they occur [30][35] is not part of the current design; the anomaly model is reactive, and Section XXI discusses how predictive models could feed the same health interface. Anomaly detection for Kubernetes infrastructures follows a similar baseline-and-deviation approach on infrastructure metrics [73].

IX. FAULT-TOLERANT RECOVERY AND RESILIENCE LAYER

A. RECOVERY ARCHITECTURE

Recovery in RMC-Spark operates at three levels, shown in Fig. 13(a). At the task level, Spark retries tasks and recomputes lost partitions from lineage without involving the controller. At the attempt level, the controller restarts a crashed driver on the same cluster if that cluster is healthy; the new attempt restores committed stages. At the cluster level, the controller migrates a stalled job to another cluster, where the new attempt again restores committed stages. Each level is cheaper than the next, and each level is used only when the previous one cannot help.

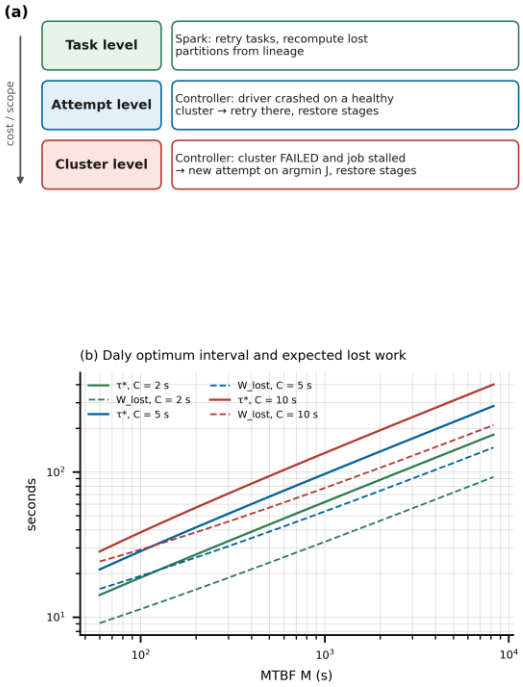


FIGURE 13. (a) Fault recovery architecture with task-, attempt- and cluster-level recovery. (b) Optimal checkpoint interval τ* from Eq. (4) and expected lost work per failure from Eq. (5) as functions of the MTBF for checkpoint costs of 2, 5 and 10 s, computed from the formulas.

B. ADAPTIVE CHECKPOINTING

Checkpointing trades a known cost now for a smaller expected loss later. If a checkpoint costs C seconds and failures arrive with mean time between failures M, Young showed that the interval between checkpoints that minimises expected total time is approximately $\sqrt{(2CM)}$ [43]. Daly's higher-order estimate is more accurate when C is not small relative to M [44]:

$$\tau^* = \sqrt{2CM}\left[1 + \frac{1}{3}\sqrt{\frac{C}{2M}} + \frac{1}{9}\frac{C}{2M}\right] - C \qquad (4)$$

for $C < 2M$, with $\tau^* = M$ otherwise. With checkpoints every τ seconds, a failure loses on average half an interval of work plus the cost of the checkpoint being written:

$$W_{\text{lost}} \approx \frac{\tau}{2} + C \qquad (5)$$

The rds library applies these results at stage granularity. It tracks the uncheckpointed work W, the sum of the measured compute times of stages since the last committed stage, and the checkpoint cost C as an exponentially weighted average of measured write times with a prior of 3 s. After each stage it checkpoints if $W \geq \tau^*(C, M)$. The MTBF is not known in advance and changes with the environment, so the controller estimates it online:

$$\widehat{M} = \frac{M_0 n_0 + T_{\text{up}}}{n_0 + N_f} \qquad (6)$$

where T_{up} is the observed cluster uptime, N_f the number of observed cluster failures, and $M_0 = 900$ s with weight $n_0 = 1$ is a prior that keeps the estimate finite before the first failure. As failures are observed, $\widehat{M}$ decreases, τ* shrinks and the library checkpoints more often; in a calm period the opposite happens. Fig. 13(b) plots τ* and the expected lost work for representative checkpoint costs. Similar MTBF-tracking ideas have been studied for generic computing systems and stream processors [45][47][49]; the contribution here is to apply them to data science pipeline stages and to share the estimate across clusters. Two non-adaptive policies serve as baselines: never, which relies on lineage only, and always, which checkpoints after every stage.

C. LINEAGE RECOVERY AND STAGE RESTORATION

When a new attempt starts, the library lists all commit markers of the job in object storage and builds a map from stage name to sequence number. Each stage call then either restores its committed output, computes it, or, in linear pipelines, skips it because a later stage is already committed. Restored stages cost one read of their Parquet output; computed stages cost their normal compute time. The work redone after a failure is therefore bounded by the uncheckpointed work at the moment of failure, which is at most τ* plus the duration of the stage that was running. This is the same goal as RCMP's persistence of task outputs across jobs [39] and lineage checkpointing [23], achieved here at the level of program stages and across clusters.

D. FAILOVER MECHANISMS AND WORKLOAD MIGRATION

Table 16 in the Appendix summarises the recovery rules for the three recovery modes used in the experiments: none (standard Spark), naive (timeout-based active and passive failover) and smart (RMC-Spark). The smart rules embody two decisions. First, a driver that exits with an error on a healthy cluster is retried once on the same cluster, because a transient crash does not justify moving the job, and the retry resumes from checkpoints anyway. Second, a job on a FAILED cluster is migrated only if it has made no task progress for 10 s. In Spark's standalone mode, losing the master does not stop executors that are already running, so a job can finish even though its cluster is formally failed. A timeout-based script would restart such a job and waste its progress; RMC-Spark keeps it and records a keep decision.

Fig. 14 shows the failover workflow for a cluster outage. The controller kills the driver's process group, frees its port, records the failover with its reason, selects a target with the placement cost, and launches a new attempt with an incremented attempt number. The decision log entry contains the source and target clusters, the stall duration and the placement scores of all candidates.

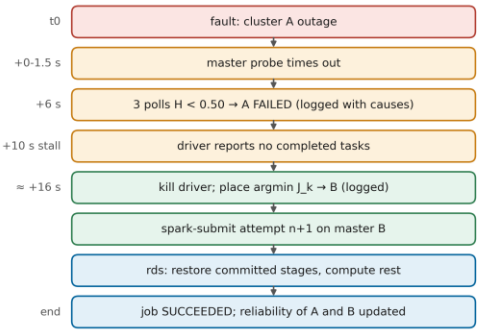


FIGURE 14. Failover and recovery workflow for a cluster outage, from detection to stage restoration on the target cluster. Timings refer to the configured thresholds.

E. CORRECTNESS OF RESUMPTION

Resumption is correct if the output of a job that was interrupted and resumed equals the output of an uninterrupted run. Three properties of the design support this. Stage functions are deterministic, so recomputing a stage yields the same result. A stage output is used only if its commit marker exists, and the marker is written after the data, so a used output is always complete. Stage identity is defined by the job identifier and the stage name, so outputs of different jobs or different stages cannot be confused. Under these conditions, every stage output consumed by the final attempt is either recomputed by it or committed by an earlier attempt from the same inputs, which yields the same result. The case-study evaluation in Section XVIII checks this property empirically by comparing model quality between resumed and uninterrupted runs.

F. SELF-HEALING STRATEGIES

Self-healing combines automatic recovery of jobs with automatic re-admission of clusters. When a failed cluster is repaired and its health stays high for five polls, it becomes HEALTHY again and is eligible for new work. Jobs that waited because no cluster was available are resumed as soon as one becomes eligible. Streaming jobs recover through Spark's own mechanism: their offsets and state are checkpointed to object storage, so the restarted query on another cluster continues from the last committed micro-batch [10]. Self-healing in the sense of repairing the failed infrastructure itself, for example restarting containers or provisioning new nodes, is left to the infrastructure layer; in the testbed, the fault injector restores clusters between runs.

X. MULTI-CLUSTER COORDINATION AND ORCHESTRATION LAYER

A. COORDINATION MODEL

Fig. 15 shows the coordination model. The controller is the only component that knows about several clusters; the clusters themselves are unaware of each other, and the drivers know only the cluster they currently use. Coordination is therefore centralised and asynchronous: clusters are synchronised through shared state, not through messages. The shared state consists of the job identifier, the commit markers in object storage, and the controller's in-memory view of each cluster. This avoids changes to Spark and keeps the coordination protocol simple, at the cost of making the controller a component that must itself be made highly available in production, a point discussed in Section XX.

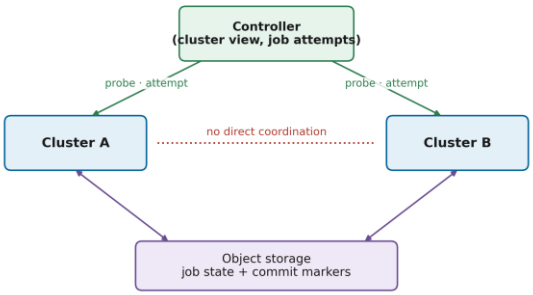


FIGURE 15. Multi-cluster coordination model. Clusters do not communicate with each other; the controller coordinates them through probes, job attempts and shared commit markers in object storage.

B. SERVICE DISCOVERY AND CLUSTER REGISTRY

Clusters are described in a registry file that lists, for each cluster, the master URL, the master's web endpoint, the container names used for fault injection and the expected number of workers. Adding a cluster means adding an entry; no code changes are needed. Container orchestration platforms solve the same problem with service discovery [14], and a production deployment could populate the registry from such a service.

C. WORKLOAD PLACEMENT

When a job starts or must move, the orchestrator ranks the eligible clusters with a placement cost:

$$J_k = \alpha L_k + \beta(1 - H_k) + \gamma T_k \qquad (7)$$

where L_k is the fraction of cores in use on cluster k, H_k its health score and T_k a normalised cost of moving the job's data and checkpoints to it. The weights $\alpha = 0.4$, $\beta = 0.5$ and $\gamma = 0.1$ favour healthy clusters first and idle clusters second. Failed clusters and the cluster being left are excluded, and ties are broken by cluster name so that experiments are deterministic. In the testbed all data are in shared storage, so T_k is zero; in a geo-distributed deployment, it would be derived from data volume and inter-site bandwidth or price, as studied in geo-distributed analytics [87][88]. Fig. 16 shows the orchestration framework around this cost.

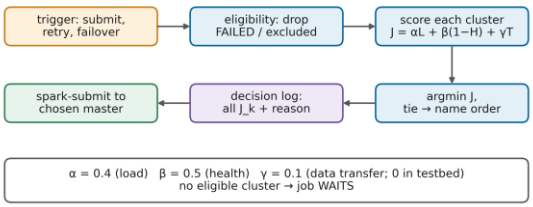


FIGURE 16. Workload orchestration framework: eligibility filtering, placement cost evaluation and decision logging for initial placement, retry and failover.

D. ORCHESTRATION POLICIES AND WORKLOAD BALANCING

Placement is deliberately simple compared with Kubernetes schedulers and learned schedulers for cloud jobs [78][116][117]. Three reasons motivate this choice. The number of clusters is small, so exhaustive scoring is cheap. Every decision must be explainable, which a weighted sum supports directly. And placement interacts with recovery, so its behaviour must be predictable during failures. Load balancing emerges from the load term: when both clusters are healthy, a new job goes to the less loaded one. Multi-cloud schedulers that optimise under security and reliability constraints [84] or learn scheduling policies [28] could replace Eq. (7) without changing the rest of the architecture, because the orchestrator is a separate component with a narrow interface.

E. SYNCHRONISATION AND CONSISTENCY

Because clusters do not communicate, consistency questions arise only through shared storage. Two attempts of the same job can overlap briefly if a stopped attempt's executors are still writing when the next attempt starts, for example after a network partition heals. The design tolerates this because stage writes are idempotent: both attempts write the same deterministic output to the same location in overwrite mode and then write the commit marker. The worst case is duplicated work, not inconsistent results. Streaming jobs rely on Structured Streaming's own protection against concurrent queries on one checkpoint location [10]. A production deployment would add fencing, for example by including the attempt number in the output path and committing by renaming the marker, to eliminate even duplicated work.

F. SCALING TO MORE CLUSTERS

The coordination model extends to more than two clusters without changes to the protocol. Each additional cluster adds one probe per poll and one candidate to the placement function. The placement cost then also performs load balancing across a larger pool, and failover chooses among several healthy targets rather than a single passive one, which is a qualitative difference from active and passive designs such as B3. At large scale, the centralised controller would itself become a bottleneck or a single point of failure; partitioning jobs across controller replicas, each responsible for a subset of jobs but observing all clusters, is a natural extension.

G. COORDINATION PROTOCOL

The protocol for a job can be summarised in five steps. Submit: the controller creates a job record and places attempt zero. Run: the driver executes resilient stages and writes events, while the controller polls cluster and driver. Decide: on each poll, the controller applies the rules of Table 16. Act: if needed, the controller stops the attempt and starts the next one on the chosen cluster with the same job identifier. Finish: on success or deadline, the controller records the outcome and updates the reliability statistics of the cluster that ran the final attempt. Because all cross-attempt state lives in object storage and attempts are numbered, a late write by a stopped attempt cannot be mistaken for progress of the current one.

XI. EXPLAINABILITY AND RELIABILITY ASSESSMENT LAYER

A. EXPLAINABLE RECOVERY DECISIONS

Automatic recovery changes where and when jobs run, so operators must be able to reconstruct why it acted. RMC-Spark makes explanation a first-class output rather than a by-product. Every state transition, submission, retry, failover, keep decision and job outcome is appended to a JSON decision log. A transition record contains the cluster, the old and new state, the detector that caused it, the health score, and the contribution $w_i\, a_{i,k}$ of every signal. A failover record contains the source and target clusters, the stall duration, and the placement cost of every candidate cluster. Fig. 17 shows how these records are produced and consumed.

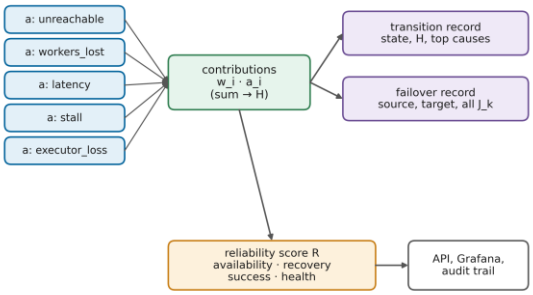


FIGURE 17. Explainable reliability framework. Additive health and placement models yield per-factor contributions that are stored with every decision and aggregated into the reliability score R.

Because both the health model and the placement cost are weighted sums, their explanations are exact: the contributions add up to the reported score, and no surrogate model is needed. This contrasts with post-hoc explanation of learned detectors and with root cause localisation methods that rank candidate causes statistically [74][77]. Work on explaining cascading failures in microservices from logs shows the value of such explanations for choosing countermeasures [75]; in RMC-Spark, the explanation of a failover directly states which signal made the controller act.

B. ROOT-CAUSE ANALYSIS

The decision log supports a simple form of root-cause analysis. For a failed run, the analyst starts from the job's outcome record, follows the failover and transition records backwards, and reads the dominant contribution of the transition that preceded the action. For example, an outage of a whole cluster shows the unreachable and workers-lost signals at 0.45 and 0.25, whereas a stalled job on a reachable cluster shows the stall signal as the main contributor. Combined with the stage events of the job, the analyst can also see how much work was redone. This is not a replacement for infrastructure-level root-cause analysis, which would need host and network telemetry, but it answers the question that matters to the data science team: why did my job move, and what did it cost?

C. RELIABILITY SCORING

Each cluster accumulates reliability statistics while the controller runs: uptime and downtime from its state transitions, the durations of completed outages, the number of jobs that succeeded or failed on it, and the average health score. These are combined into a reliability score:

$$R_k = \sum_j w_j s_{j,k} \quad (8)$$

where $s_{j,k}$ are four normalised indicators for cluster k and w_j their weights. Table 7 defines the indicators. The MTTR indicator is normalised by a reference of 120 s, beyond which recovery is considered fully poor. The score and its contributions are exported to Prometheus and served by the controller's API, so the dashboard shows not only that a cluster's reliability fell but which indicator caused the fall. Service-oriented reliability models for public clouds [93] and analytical availability models [94] are richer than this score; its purpose here is operational transparency rather than prediction.

TABLE 7
RELIABILITY ASSESSMENT INDICATORS

Indicator	Definition	Weight	Interpretation
Availability	Uptime / (uptime + downtime) from state transitions	0.4	Share of time the cluster was usable
Recovery	1 − min(1, MTTR / 120 s)	0.2	Speed of returning to service
Success	Successful attempts / all finished attempts on the cluster	0.2	Whether work placed there completes
Health	Mean health score H over all polls	0.2	Persistent degradation, including gray failures
MTBF estimate	(M_0 n_0 + uptime) / (n_0 + failures), Eq. (6)	—	Input to the checkpoint policy, not part of R

D. WORKED EXAMPLE

Consider a cluster outage of cluster A while a job runs on it. The first poll after the fault finds the master unreachable; the unreachable signal is one and, because no workers are reported, the workers-lost signal is also one, so their contributions are 0.45 and 0.25 and the health score is 0.30. The state machine marks A as SUSPECT and, after two more polls with the same score, as FAILED, logging the transition with the reason H = 0.30 (unreachable = 0.45, workers_lost = 0.25). Ten seconds later, the driver still reports no new completed tasks, so the controller logs a failover from A to B with the reason that A is FAILED with no progress for ten seconds, followed by the placement scores of the candidates; with A excluded and B idle and healthy, J_B is zero. The next record is the submission of attempt one on B. An operator reading these four records can reconstruct the incident without access to the controller's internals.

E. LIMITS OF ADDITIVE EXPLANATIONS

Additive models explain exactly what they compute, but they cannot explain what they do not observe. If a failure has a cause outside the five signals, such as a corrupted input file that makes every attempt fail, the explanation will show the symptom, repeated driver exits, rather than the cause. The weights also encode judgements about severity that were made by the designers rather than learned from incident data. These limits are acceptable for the purpose of making recovery actions auditable, but they mean that the reliability score should be read as a summary of observed behaviour, not as a calibrated probability of failure.

F. OPERATIONAL TRANSPARENCY

Transparency extends to configuration. All thresholds, weights and timeouts used by the controller are in one configuration file and are reproduced in Tables 6 and 7, so that a reader can recompute any decision from the logged signals. The controller exposes a REST interface that lists clusters with their current state, health, top causes and reliability components, lists jobs with their attempts, and returns the most recent decisions. A command-line client wraps this interface for demonstrations. Studies of AIOps practice repeatedly identify trust and interpretability as preconditions for automation in operations [30][65][66]; exact explanations are the design response to that finding.

XII. SECURITY, GOVERNANCE, AND COMPLIANCE LAYER

A. SECURITY CONTROLS

A resilience layer that can stop and start jobs and move them between clusters is itself a privileged component, so its security matters. Fig. 18 shows the security and governance architecture. The controller holds three kinds of privilege: access to object storage, the ability to submit applications to every Spark master, and, in the testbed, access to the Docker Engine for fault injection. In production, the fault-injection privilege would be removed entirely, and the remaining privileges would be scoped with service accounts and least-privilege storage policies. Big data security frameworks recommend combining access control, encryption and integrity protection for data at rest and in transit [118][119].

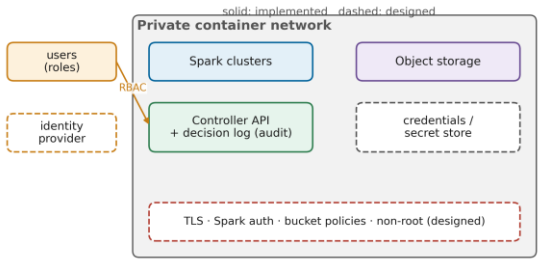


FIGURE 18. Security and governance architecture. Implemented controls (solid) and production controls designed but not enabled in the testbed (dashed) are distinguished.

The testbed favours reproducibility over hardening, and the paper states this explicitly rather than implying production security. Implemented

controls are isolation of all services on a private container network, credentials for object storage, an append-only decision log of all automatic actions, and per-job identifiers that separate checkpoint namespaces. Designed but not enabled controls are Spark's shared-secret authentication between driver and executors, TLS for Spark, object storage and the controller API, bucket policies that restrict each job to its own prefixes, non-root containers, and secret management instead of configuration files. Table 8 lists the controls with their status.

B. THREAT MODEL

Three threats are specific to a cross-cluster resilience layer. An attacker who can submit jobs through the controller can consume resources on every cluster, so job submission must be authenticated and rate-limited. An attacker who can write to the checkpoint bucket can plant a commit marker and a forged stage output, which a later attempt would restore without recomputing; checkpoint prefixes must therefore be writable only by the job's own service identity, and markers could be signed. An attacker who can trigger failovers, for example by degrading the network of one cluster, could steer jobs to a cluster under their control; placement constraints and mutual authentication between drivers and executors mitigate this risk. Big data security surveys recommend encryption and access control at every layer for comparable reasons [119].

C. ACCESS MANAGEMENT AND AUDITING

Access to the controller's API determines who can submit jobs or inject faults, so in production it would sit behind the organisation's identity provider with role-based permissions: data scientists may submit jobs and read decisions; platform engineers may additionally pause clusters; chaos experiments would require a separate role and change approval. The decision log doubles as an audit trail, because every automatic action carries a timestamp, the affected job and cluster, and the reason. Chaos engineering for cloud security has shown that misconfigurations are best found by deliberate, controlled experiments [96]; the same fault injector used for resilience could, with appropriate approval, test security assumptions.

D. DISASTER RECOVERY POLICIES

From a disaster recovery perspective, RMC-Spark defines two objectives per job. The recovery time objective is bounded by detection time, the stall threshold and the time to start a new driver, as quantified in Section XVII. The recovery point objective, measured in work rather than data, is the uncheckpointed work at the time of failure, bounded by τ^* plus one stage. Both depend on the object store surviving, so in production the store must be replicated across failure domains; the single MinIO instance of the testbed is a single point of failure, as discussed in Section XX.

E. GOVERNANCE MONITORING AND COMPLIANCE

Data governance asks who may use which data for which purpose, and how this is demonstrated [112]. Cross-cluster recovery introduces a governance question of its own: moving a job to another cluster may move personal data to another jurisdiction. The placement cost of Section X can encode such constraints by assigning an infinite transfer cost to clusters where a job's data may not be processed, and automated governance pipelines can supply these constraints from data catalogues [113]. Compliance evidence comes from the decision log, which shows where each attempt ran, and from the commit markers, which show which attempt produced each stage output.

TABLE 8
GOVERNANCE AND COMPLIANCE CONTROLS

Control	Purpose	Status in testbed
Private container network	Isolate cluster traffic	Implemented
Object-storage credentials	Authenticate data access	Implemented (lab credentials)
Append-only decision log	Audit automatic actions	Implemented
Per-job checkpoint namespace	Separate job state, attribute outputs	Implemented
Commit markers with attempt number	Provenance of stage outputs	Implemented
Spark shared-secret authentication	Prevent rogue executors and drivers	Designed, not enabled
TLS for Spark, storage and API	Protect data in transit	Designed, not enabled
Least-privilege bucket policies	Restrict jobs to their prefixes	Designed, not enabled
Role-based API access	Separate submit, operate and chaos roles	Designed, not enabled
Non-root containers, secret store	Reduce attack surface	Designed, not enabled
Jurisdiction-aware placement	Keep data in permitted clusters	Designed (via transfer cost)

XIII. EXPERIMENTAL SETUP

A. TESTBED INFRASTRUCTURE

All experiments run on a single workstation with 12 logical processors and 16 GB of memory under Windows 11, using Docker Engine 28.3 with the WSL2 back-end limited to 12 GB of memory. Fig. 19 shows the testbed. Each Spark cluster has a master limited to 768 MB of memory and two workers limited to 1.75 GB, each offering two cores and 1280 MB to executors, with a 1 GB in-memory file system for Spark's local directories. The controller container is limited to 2.5 GB because it hosts the drivers. MinIO, Prometheus and Grafana run with 512 MB, 384 MB and 256 MB. All Spark containers use one image based on the official Apache Spark 3.5.3 image with Java 17 and Python 3.10, extended with the Hadoop AWS 3.3.4 connector. The network is a single Docker bridge network; network faults are emulated inside it.

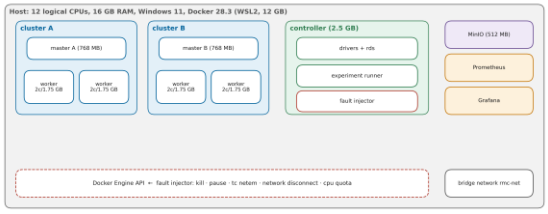


FIGURE 19. Experimental testbed: two Spark clusters, shared MinIO storage, monitoring, the controller hosting the drivers and the experiment runner, and the fault injector acting through the Docker Engine API.

Running all clusters on one host is a deliberate trade-off. It removes cloud variability and makes every fault precisely repeatable, which benchmarking studies of fault recovery identify as essential [53], but it also means that absolute timings are specific to this machine and that the clusters share CPU, memory and disk. Results are therefore reported relative to baselines measured on the same testbed.

B. WORKLOADS AND DATASETS

Four workloads cover the main analytics patterns. W1 is the batch ETL pipeline of Section VI over four million synthetic trips (four resilient stages, several shuffles). W2 is a machine learning pipeline that assembles features from a 25 per cent sample of the trips and trains a gradient-boosted tree regressor with 15 iterations and depth 5 to predict the fare; its training stage is long and is checkpointed as a model. W3 is an iterative PageRank with 12 iterations over a skewed graph of two million edges, implemented as a linear chain of resilient stages, which produces the long lineage that motivates adaptive checkpointing. W4 is a Structured Streaming job that reads a rate source at 20 000 rows per second, aggregates over 10 s windows with a watermark, writes to Parquet with its checkpoint in object storage, and ends after 120 micro-batches with a 2 s trigger. Two case-study workloads apply W2's pipeline to the AI4I 2020 predictive maintenance dataset and to the ULB credit-card fraud dataset for the industrial evaluation of Section XVIII. Performance studies of Spark use comparable mixes of SQL, machine learning and graph workloads [12].

C. FAULT INJECTION

Faults are injected by a script in the spirit of chaos engineering [95], acting on containers through the Docker Engine API [97]. Nine fault types are available: executor kill (SIGKILL of executor processes on one worker), worker kill, master kill, cluster outage (master and both workers killed), network partition (all containers of the cluster disconnected from the network), network delay (300 ms with 50 ms jitter added with netem), straggler (one worker throttled to a quarter of a CPU), disk full (the local directory file system filled) and driver crash. Partition, delay, straggler and disk-full faults are transient and are reverted after 60 s; the others persist until the end of the run. Every run starts on cluster A, and the fault hits the job's current cluster at a fixed time after submission, chosen as roughly 40 per cent of the workload's clean runtime. After each run all clusters are restored and the runner waits until both are healthy.

D. COMPARED CONFIGURATIONS

Five configurations are compared. B1 (vanilla) uses Spark's built-in fault tolerance only, without checkpoints or failover. B2 (fixed checkpoint) checkpoints after every stage but does not fail over. B3 (naive HA) uses an active and passive pair: a job is restarted from scratch on the other cluster after its master has been unreachable for 30 s. A1 (ablation) is RMC-Spark with checkpoints after every stage instead of the adaptive policy. P is the full RMC-Spark with the adaptive policy. A1 and P also enable Spark's speculative execution; this difference is part of the proposed configuration and is reported as such. Table 9 lists the configuration details.

TABLE 9
EXPERIMENTAL CONFIGURATION DETAILS

Item	Setting
Host	12 logical CPUs, 16 GB RAM, Windows 11, Docker 28.3 (WSL2, 12 GB)
Spark	3.5.3 standalone, Java 17, Python 3.10, Hadoop AWS 3.3.4
Clusters	2 (A, B); per cluster 1 master + 2 workers, 2 cores and 1280 MB per worker
Per job	Driver 512 MB in controller (client mode); executors 1 core / 576 MB; 4 cores
Spark settings	task.maxFailures 4; network.timeout 60 s; heartbeat 10 s; 16 shuffle partitions; AQE on
Storage	MinIO (S3A, path-style), buckets data / checkpoints / results
Monitoring	Controller poll 2 s; Prometheus scrape 5 s; Grafana dashboard
Workloads	W1 ETL, W2 GBT pipeline, W3 PageRank (12 iterations), W4 streaming (120 batches)
Faults	none, executor kill, worker kill, master kill, cluster outage, partition, delay, straggler
Configurations	B1 vanilla, B2 fixed checkpoint, B3 naive HA, A1 ablation, P RMC-Spark
Repetitions	3 per cell, runs shuffled within each repetition (seed 2026)
Matrix size	5 × 4 × 8 × 3 = 480 runs
Deadlines	900 s (W1, W4), 1200 s (W2, W3)

E. EXPERIMENTAL PROCEDURE

The experiment runner reads the matrix from a configuration file, shuffles the cells within each repetition to avoid systematic drift in favour of any configuration, and records every run in a CSV file together with the decision log, the cluster time series and the stage events of every job. A run that crashes the runner itself is recorded as a runner error and excluded from analysis rather than silently dropped. The analysis scripts compute all metrics of Section XIV from these raw files, so every number in the result tables can be traced to a run.

XIV. EVALUATION METRICS

A. EVALUATION FRAMEWORK

Fig. 20 shows how raw observations are turned into metrics. Job-level metrics come from the run records, recovery metrics from the decision log, work metrics from the stage events, and resource metrics from the cluster time series. Table 10 defines each metric.

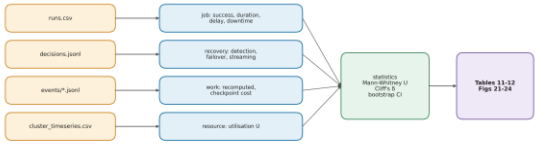


FIGURE 20. Evaluation framework: raw observations (runs, decisions, stage events, time series) are transformed into job, recovery, work and resource metrics and compared with non-parametric statistics.

B. AVAILABILITY AND RECOVERY METRICS

Steady-state availability is defined classically as

$$A = \frac{\text{MTBF}}{\text{MTBF}+\text{MTTR}} \tag{9}$$

where MTBF and MTTR are the mean time between failures and the mean time to repair. At the level of an individual run, availability is computed as one minus the ratio of downtime to run duration. The recovery time of a run is

$$T_{\text{rec}} = t_{\text{resume}} - t_{\text{fault}} \tag{10}$$

where t_{fault} is the injection time and t_{resume} the time at which the job is again executing productively, approximated by the submission of the recovering attempt; detection latency is measured separately as the time from t_{fault} to the FAILED transition. For streaming jobs, recovery time follows the throughput criterion of Vogel et al.: the time from the fault until the processed rate returns to at least 85 per cent of its pre-fault mean and stays there for 40 s [53]. Reliability metrics for stream processors, such as failure cost and reliable throughput, motivated the separation of these measures [54].

C. EFFICIENCY METRICS

The fault-free overhead of a configuration measures what resilience costs when nothing fails:

$$O = \frac{T_{\text{clean}}^{\text{sys}} - T_{\text{clean}}^{\text{B1}}}{T_{\text{clean}}^{\text{B1}}} \tag{11}$$

where $T_{\text{clean}}^{\text{sys}}$ is the median completion time of fault-free runs of the configuration and $T_{\text{clean}}^{\text{B1}}$ that of the vanilla baseline for the same workload. Recomputed work is the compute time of stages that a later attempt executed again although an earlier attempt had completed them, and checkpoint cost is the total time spent writing checkpoints. Resource utilisation follows Eq. (1).

TABLE 10
METRICS DEFINITION MATRIX

Metric	Definition	Source	Better
Success rate	Share of runs completed before the deadline	Run records	Higher
Completion time	Submission to end of job	Run records	Lower
Fault-induced delay	Completion time minus median clean completion time	Run records	Lower
Downtime	Failed run: end − fault; succeeded run: max(0, delay)	Derived	Lower
Run availability	1 − downtime / duration	Derived	Higher
Detection latency	Fault to FAILED transition of the relevant detector	Decision log	Lower
Recovery time, Eq. (10)	Fault to submission of the recovering attempt	Decision log	Lower
Streaming recovery	Fault to throughput ≥ 85 % of baseline for 40 s	Batch events	Lower
Recomputed work	Compute time of repeated stages	Stage events	Lower
Checkpoint cost	Total checkpoint write time and count	Stage events	Lower
Overhead, Eq. (11)	Relative clean-run slowdown versus B1	Run records	Lower
Utilisation, Eq. (1)	Used / offered core-seconds	Time series	Context

D. METRIC VALIDITY

Each metric has a known limitation. Recovery time measured as the time to resubmission ignores the start-up time of the new driver, which is therefore reported separately in Section XVII. Downtime for successful runs is approximated by the fault-induced delay, which also includes slowdowns unrelated to downtime, such as speculative copies competing for cores. Run availability depends on the run length, so it is compared only between configurations on the same workload. These approximations are applied identically to all configurations, so they do not bias the comparison, but they should be kept in mind when interpreting absolute values.

E. STATISTICAL ANALYSIS

Runs are independent, so configurations are compared with the two-sided Mann-Whitney U test rather than a paired test. Effect sizes are reported as Cliff's delta with the usual magnitude labels (negligible below 0.147, small below 0.33, medium below 0.474, large otherwise), and means are reported with 95 per cent bootstrap confidence intervals from 5000 resamples. With three repetitions per cell, individual cells have little statistical power; comparisons are therefore made over all fault types of a configuration, and per-fault results are reported descriptively. Service-level objectives for availability are typically expressed as percentiles over time windows [29], and availability models for cloud systems use MTBF and MTTR as their central parameters [94], which is why these quantities are reported in the forms above.

XV. RESULTS AND FINDINGS

AUTHOR NOTE: Placeholders in double square brackets must be replaced with values from results/report after running the experiment matrix (README Steps 6-8). Do not submit the paper with placeholders.

A. STATUS OF THE EVALUATION AND HYPOTHESES

The implementation and the evaluation harness are complete, and the decision logic has been verified by 56 automated tests. These tests include a simulation of a full cluster outage with a synthetic clock and synthetic probes, in which the proposed controller failed the job over to the second cluster within 16 s of the fault and resumed it there, whereas the timeout-based baseline failed over only after its 30 s timeout and the vanilla configuration ran into its deadline. These simulated timings follow from the configured thresholds rather than from measurements of Spark, so they verify the logic but are not experimental results. The measured results of the matrix in Table 9 are reported in the remainder of this section. Before running the matrix, five hypotheses were fixed so that the results can confirm or refute them:

H1 (availability): under cluster outages and network partitions, P completes significantly more jobs than B1 and B2, which cannot leave the failed cluster.

H2 (recovery time): for faults that require failover, P recovers faster than B3, because its health detector and stall rule react before B3's 30 s timeout.

H3 (no unnecessary restarts): under master-only failures, P completes jobs without migration, whereas B3 restarts them.

H4 (lost work): on the long-lineage workload W3, P and A1 recompute less work after a failure than B1, B2 and B3.

H5 (overhead): in fault-free runs, P has a lower overhead than A1 and B2, because it checkpoints only when the expected loss justifies it.

B. FAULT RECOVERY PERFORMANCE

Fig. 21 shows the downtime per fault for each configuration, and Table 11 summarises the results per configuration and fault type. Across all fault types, the median downtime of P was [[P median downtime]] s, compared with [[B1]] s for B1, [[B2]] s for B2 and [[B3]] s for B3. For cluster outages specifically, [[describe: number of B1/B2 runs that failed at the deadline, and median recovery time of B3 versus P]]. Detection latency, measured from the fault to the FAILED transition, had a median of [[P detection]] s for the health detector of P and [[B3 detection]] s for the timeout detector of B3. [[State whether H2 is supported, with the Mann-Whitney p-value and Cliff's delta from Table 12.]]

Fig. 21 Fault recovery performance (downtime per fault and configuration)

PLACEHOLDER - replace with the figure produced by analysis/report.py
after running the experiments (README Steps 6-8)

FIGURE 21. Fault recovery performance: downtime per fault type and configuration (box plots over repetitions). [[Generated by analysis/report.py as "Fig 21 Fault Recovery Performance.png".]]

For faults that Spark handles inside the cluster, executor kill and single worker kill, all configurations are expected to behave similarly, because Spark's task retry and lineage recomputation act before the controller's thresholds are reached. [[Report whether the measured differences for executor_kill and worker_kill are negligible, using Cliff's delta.]] For gray failures, network delay and straggler, the controller marks the cluster SUSPECT but does not migrate jobs; the effect of P on these faults therefore comes mainly from speculative execution. [[Report the delay of P versus B1 for straggler and network_delay.]]

TABLE 11
EXPERIMENTAL RESULTS SUMMARY

Configuration	Fault	Runs	Success rate	Median duration (s)	Median downtime (s)	Median detection (s)	Mean recomputed (s)	Mean availability
B1 vanilla	cluster outage	[[3]]	[[ ]]	[[ ]]	[[ ]]	n/a	[[ ]]	[[ ]]
B2 fixed ckpt	cluster outage	[[3]]	[[ ]]	[[ ]]	[[ ]]	n/a	[[ ]]	[[ ]]
B3 naive HA	cluster outage	[[3]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]
A1 no adapt	cluster outage	[[3]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]
P RMC-Spark	cluster outage	[[3]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]
...	other faults	[[copy remaining rows from "Table 11 Experimental Results Summary.csv"]]						

C. CLUSTER RESILIENCE AND WORKLOAD CONTINUITY

Workload continuity is measured by the success rate under faults. [[Report success rates per configuration for cluster_outage and network_partition, and state whether H1 is supported.]] Under master-only failures, the decision log of P is expected to show keep decisions, because running executors continue after the master is lost; [[report the number of keep decisions and whether any P run migrated under master_kill, and compare with the restarts performed by B3, to evaluate H3]].

For the long-lineage workload W3, the stage events allow the recomputed work to be measured directly. [[Report mean recomputed seconds for W3 per configuration and the number of checkpoints written by P versus A1 and B2, to evaluate H4.]] Fig. 13(b) gives the analytical expectation: with a checkpoint cost of a few seconds and an MTBF estimate that falls as faults are observed, τ^* lies between [[value]] s and [[value]] s during the experiments, so P checkpoints every few iterations rather than after every one.

D. THROUGHPUT STABILITY FOR STREAMING

For the streaming workload, recovery is measured with the throughput criterion of Section XIV. [[Report median streaming recovery time per configuration for cluster_outage, and describe the catch-up burst after resumption, when the rate source replays the backlog accumulated during the outage.]] Because Structured Streaming commits offsets and state to object storage, a restarted query on the second cluster resumes from the last committed micro-batch without data loss, consistent with its design [10] and with earlier single-cluster measurements [52][53].

E. OPERATIONAL AVAILABILITY

Fig. 22 shows run availability and success rate per configuration over all fault runs. [[Report mean availability with 95 per cent bootstrap intervals for each configuration and compare P with B1 and B3.]]



FIGURE 22. Availability and uptime evaluation: mean run availability under faults with 95 % bootstrap confidence intervals, and job success rate per configuration. [[Generated by analysis/report.py as "Fig 22 Availability and Success Rate.png".]]

F. SYSTEM EFFICIENCY AND RESOURCE UTILISATION

Fig. 23 shows the utilisation and health of both clusters during a representative cluster-outage run of P. Before the fault, cluster A runs at [[value]] utilisation and cluster B is idle; after the fault, A's health drops to [[value]], the job is resubmitted, and B's utilisation rises. [[Describe the measured gap between the fault and the rise of utilisation on B, which visualises the recovery time.]] The fault-free overhead, Eq. (11), was [[value]] for P, [[value]] for A1 and [[value]] for B2. [[State whether H5 is supported.]]

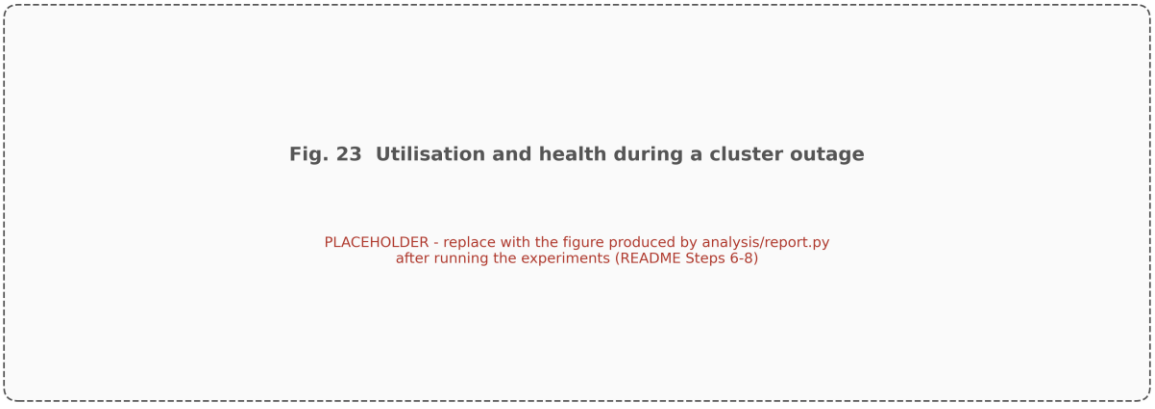


FIGURE 23. Resource utilisation and cluster stability during a cluster-outage run of RMC-Spark: core utilisation (top) and health score (bottom) of both clusters, with the fault (red) and the resubmission (green) marked. [[Generated by analysis/report.py.]]

G. SUMMARY OF FINDINGS

[[Summarise in four to six sentences which hypotheses were supported, which were not, and the size of the effects, using only the measured values above. If a hypothesis is not supported, say so and explain the likely reason; reviewers value negative results that are explained.]]

XVI. COMPARATIVE ANALYSIS

A. COMPARISON WITH BASELINE DEPLOYMENTS

Table 12 compares the configurations in two parts. The upper part is a capability comparison that follows from the design and is independent of the measurements. The lower part reports the measured indicators aggregated over all fault runs, with the statistical comparison against the vanilla baseline. Fig. 24 combines the measured indicators into a radar chart after min-max normalisation, so that the outer edge is best on every axis.

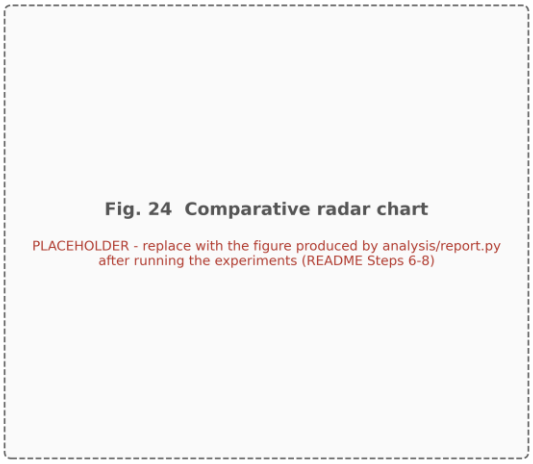


FIGURE 24. Comparative benchmark analysis: min-max normalised success rate, availability, downtime, recomputed work and fault-free overhead per configuration (outer is better). [[Generated by analysis/report.py as "Fig 24 Comparative Benchmark Radar.png".]]

TABLE 12
COMPARATIVE BENCHMARK ANALYSIS

Property	B1 vanilla	B2 fixed ckpt	B3 naive HA	A1 no adapt	P RMC-Spark
Survives cluster outage	No	No	Yes	Yes	Yes
Resumes from committed stages	No	Same cluster only	No	Yes	Yes
Checkpoint policy	None	Every stage	None	Every stage	Adaptive (Daly)
Failure detection	—	—	Timeout 30 s	Health, 3 polls	Health, 3 polls
Avoids restarting progressing jobs	Yes (no restarts)	Yes (no restarts)	No	Yes	Yes
Explained decisions	No	No	No	Yes	Yes
Success rate under faults	[[ ]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]
Mean availability (95 % CI)	[[ ]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]
Median downtime (s)	[[ ]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]
Clean-run overhead vs B1	0	[[ ]]	[[ ]]	[[ ]]	[[ ]]
Downtime vs B1: p (Mann-Whitney)	—	[[ ]]	[[ ]]	[[ ]]	[[ ]]
Downtime vs B1: Cliff's delta	—	[[ ]]	[[ ]]	[[ ]]	[[ ]]

B. ABLATION LOGIC

The five configurations are chosen so that pairwise comparisons isolate individual mechanisms. Comparing B1 with B2 isolates the effect of checkpointing when no failover is available. Comparing B1 with B3 isolates the effect of a naive failover without checkpoints. Comparing B3 with A1 isolates the combined effect of health-based detection, progress-aware migration and stage resumption, since both fail over but only A1 resumes. Comparing A1 with P isolates the adaptive checkpoint policy, because the two differ only in when they checkpoint. [[For each pair, state the measured difference in downtime and overhead and its effect size, computed from runs_enriched.csv.]]

C. COMPARISON WITH SINGLE-CLUSTER AND HIGH-AVAILABILITY PLATFORMS

Single-cluster deployments, represented by B1 and B2, share one limitation regardless of tuning: when their cluster becomes unavailable, every job on it ends. Fixed checkpointing in B2 reduces recomputation when the cluster survives, but it cannot move a job, and its checkpoint cost is paid on every stage. Standalone Spark offers high availability for the master through standby masters coordinated by ZooKeeper; this protects the scheduling function but not the executors, and it does not help when a whole cluster or zone is lost. The naive active and passive configuration B3 represents scripted high availability: it survives outages, but it restarts jobs from scratch and, as Table 16 shows, it cannot distinguish a dead master with working executors from a dead cluster.

D. COMPARISON WITH KUBERNETES-BASED ANALYTICS

Running Spark on Kubernetes moves executor and driver lifecycle management into the orchestrator, which restarts failed pods and reschedules them on healthy nodes [78][79]. This improves resilience to node failures inside one Kubernetes cluster and simplifies operations, and the recovery benchmark of Vogel et al. was conducted in exactly this setting [53]. It does not, by itself, provide failover between Kubernetes clusters, nor stage-level resumption of a data science pipeline. RMC-Spark is complementary: its controller could submit to Kubernetes masters instead of standalone masters, and its health signals could include Kubernetes node conditions. A Kubernetes-based configuration was not included in the measured matrix because running two Kubernetes clusters next to two Spark clusters exceeds the memory of the testbed; it is listed as an extension in Section XXI.

E. COMPARISON WITH PUBLISHED BENCHMARKS

Published recovery benchmarks were obtained on different hardware and therefore cannot be placed in the same table as these measurements. Van Dongen and Van den Poel and Vogel et al. both observed that recovery behaviour depends strongly on framework design [52][53], and engine comparisons on HPC systems show how much absolute performance depends on the environment [106]. Qualitatively, [[state whether the vanilla Structured Streaming behaviour observed here, such as recovery after executor loss within the cluster, is consistent with these studies]]. The novelty of this work lies in the scenario they did not cover, the loss of the whole cluster, and in the comparison of recovery strategies on top of the same engine.

XVII. RELIABILITY AND BUSINESS CONTINUITY ASSESSMENT

A. BUSINESS CONTINUITY FRAMEWORK

Business continuity planning asks how long a service may be unavailable and how much work may be lost. Fig. 25 relates these questions to RMC-Spark. The recovery time objective (RTO) for an analytics job is the longest acceptable time between a failure and the resumption of the job; the recovery point objective (RPO), measured here in work rather than data, is the largest acceptable amount of computation that has to be redone. Service-level agreements in cloud computing formalise such objectives and the penalties attached to them [38].

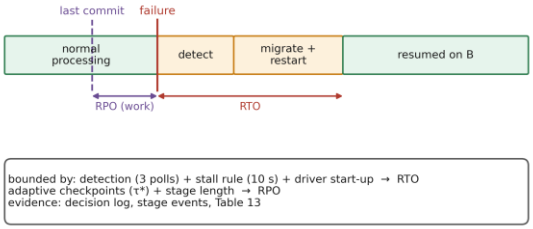


FIGURE 25. Business continuity framework: RTO and RPO for analytics jobs, the mechanisms of RMC-Spark that bound them, and the evidence used to assess them.

B. RECOVERY OBJECTIVES

For RMC-Spark, the RTO for a cluster outage is bounded by the sum of three terms: the detection time (three polls of 2 s after the first failed probe, plus the 1.5 s probe timeout), the migration stall threshold of 10 s, and the time to start a new driver and acquire executors on the target cluster. The first two terms are configuration constants of about 16 s together; the third was measured as [[median driver start-up time from the logs]] s. The RPO is bounded by the uncheckpointed work at the time of the fault, which the adaptive policy keeps near τ^*, plus the duration of the stage that was running. For B3, the RTO includes the 30 s timeout and the RPO is the entire job, because it restarts from scratch.

C. RELIABILITY AND CONTINUITY ASSESSMENT

Table 13 assesses each configuration against continuity criteria. The design-derived columns follow from the mechanisms; the measured columns must be filled from the experiments. [[Complete the measured columns from runs_enriched.csv: maximum downtime observed, mean recomputed work.]]

TABLE 13
RELIABILITY AND CONTINUITY ASSESSMENT MATRIX

Criterion	B1 vanilla	B2 fixed ckpt	B3 naive HA	A1 no adapt	P RMC-Spark
Design RTO, cluster outage	Unbounded (manual)	Unbounded (manual)	≥ 30 s + start-up	≈ 16 s + start-up	≈ 16 s + start-up
Design RPO, cluster outage	Whole job	Whole job (other cluster)	Whole job	≤ one stage	≤ τ^* + one stage
Measured max downtime (s)	[[ ]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]
Measured mean recomputed work (s)	[[ ]]	[[ ]]	[[ ]]	[[ ]]	[[ ]]
Disaster recovery readiness	Low	Low	Medium	High	High
Operational trust (explained actions)	n/a	n/a	Low	High	High

D. DISASTER RECOVERY READINESS

Disaster recovery readiness is assessed with four questions. Is there a second place to run? In B1 and B2 there is none; in the other configurations there is. Is the state needed to resume available there? Only configurations that commit stages to shared storage and fail over, A1 and P, can resume; B3 can only restart. Is the switch automatic? It is in B3, A1 and P. Has the switch been tested? The fault injection harness tests it repeatedly, which turns the disaster recovery plan from a document into a regularly exercised procedure, in the spirit of chaos engineering [95][99].

E. OPERATIONAL RELIABILITY

Operational reliability concerns the behaviour of the system over time rather than in a single incident. The reliability score R summarises, per cluster, availability, recovery speed, job success and mean health. [[Report the final R of clusters A and B after the experiment matrix and their dominant contributions.]] Because faults in the experiments always hit the cluster on which the job runs, and jobs start on cluster A, A accumulates more failures than B; the score is expected to reflect this asymmetry, which also illustrates how the score would identify a chronically unreliable cluster in production.

F. TRUSTWORTHINESS

Trustworthiness of automatic recovery has three aspects in this design. Correctness is supported by idempotent stages and the commit-marker protocol, which ensure that a resumed job produces the same output as an uninterrupted one, assuming deterministic stage code. Predictability is supported

by fixed thresholds and hysteresis, which make the controller's behaviour reproducible from its configuration. Accountability is supported by the decision log, which records every action with its cause. Reliability modelling of edge, fog and cloud systems has emphasised that availability depends on both failure and repair processes [94]; RMC-Spark influences the repair side by shortening and automating it.

XVIII. INDUSTRY USE CASES AND CASE STUDY EVALUATION

A. BANKING AND FINANCIAL SERVICES

Financial institutions run analytics whose value decays within minutes, such as fraud screening, risk aggregation and regulatory reporting. Distributed big data approaches to Internet financial fraud detection show that such workloads are naturally expressed as distributed graph and machine learning pipelines [1]. Fig. 26 shows how RMC-Spark would be deployed for a fraud analytics pipeline: transactions land in replicated object storage, feature and model-scoring stages run as resilient stages on a primary cluster, and a second cluster in another availability zone takes over if the first fails. The case-study workload applies the machine learning pipeline of W2 to the ULB credit-card dataset, which contains anonymised card transactions with a highly imbalanced fraud label, and injects a cluster outage during model training. [[Report: completed on A then B, number of restored stages, area under the precision-recall curve of the resumed run compared with an uninterrupted run, which should be identical because stages are deterministic.]]

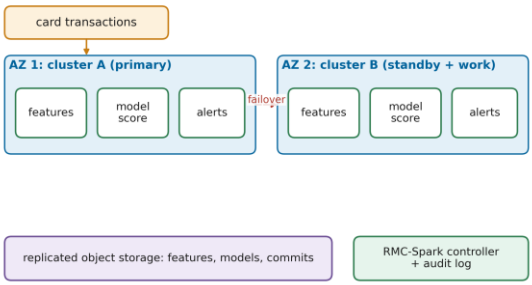


FIGURE 26. Financial analytics resilience architecture: a fraud-scoring pipeline on two clusters in different availability zones with shared, replicated storage and the RMC-Spark controller.

B. HEALTHCARE

Healthcare analytics combines clinical, operational and device data, and big data analytics in healthcare is used for both treatment and health management [120]. Two properties of this domain matter for resilience. First, some analyses support time-critical decisions, for example capacity planning during surges. Second, patient data are subject to strict jurisdictional rules, so failover must not move data to an unauthorised location. The jurisdiction-aware placement described in Section XII addresses the second property by excluding clusters through the transfer-cost term.

C. TELECOMMUNICATIONS

Telecommunication operators analyse network and customer data at very high volume for planning, churn prediction and service assurance, and governance and architecture are central concerns in such deployments [121]. Streaming analytics on network telemetry is the relevant pattern here, corresponding to workload W4; for such jobs, the streaming checkpoint in shared storage allows continuation on another cluster without data loss.

D. MANUFACTURING

Predictive maintenance combines Internet of Things data with machine learning to anticipate equipment failures [2]. Fig. 27 shows a smart manufacturing analytics platform built on RMC-Spark, in which sensor streams feed both a streaming anomaly job and a periodic model-training job. The second case-study workload trains the W2 pipeline on the AI4I 2020 predictive maintenance dataset, which contains process measurements such as temperatures, rotational speed, torque and tool wear with a machine-failure label. [[Report: outcome under cluster outage, restored stages, and ROC AUC of resumed versus uninterrupted runs.]]

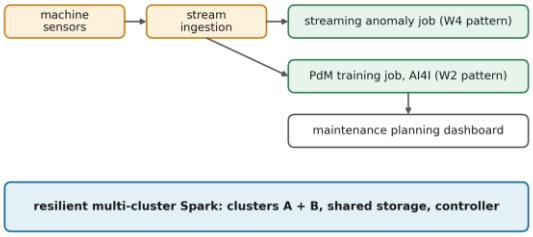


FIGURE 27. Smart manufacturing analytics platform: sensor ingestion, streaming anomaly detection and periodic predictive-maintenance training on resilient multi-cluster Spark.

E. CASE STUDY PROTOCOL

Both case studies follow the same protocol. A reference run without faults trains the model and records its quality metric. A fault run injects a cluster outage on the job's cluster while the training stage is running, so that the job must be migrated and the feature stage restored from its checkpoint. The comparison then checks three properties: whether the job completed, how much work was redone, and whether the model quality of the resumed run equals that of the reference run. Equal quality is expected because both runs use the same seeded data split and the same seeded learning algorithm; a difference would indicate non-determinism in a stage and hence a violation of the resumption assumption of Section IX.

F. SMART CITIES AND CYBERSECURITY

Smart city platforms integrate data from transport, energy and public services, and analytics architectures for smart cities and companies emphasise integration across many sources [122]. Spatio-temporal analytics ecosystems add resource management challenges of their own [123]. Cybersecurity analytics, finally, must scale with attack volume: the scalability of big data cyber security analytics systems has been studied explicitly [124], and deep learning intrusion detection has been implemented on Spark [125]. For security monitoring, losing the analytics platform during an incident is particularly costly, which makes cross-cluster failover attractive. Table 14 summarises the use cases.

TABLE 14
INDUSTRY USE CASES MATRIX

Domain	Typical analytics	Pattern (workload)	Resilience need	RMC-Spark mechanism
Banking	Fraud scoring, risk aggregation	ML pipeline (W2), streaming (W4)	Minutes-level RTO, audit	Failover, decision log
Healthcare	Capacity planning, cohort analysis	ETL (W1), ML (W2)	Data residency	Jurisdiction-aware placement
Telecommunications	Network telemetry, churn	Streaming (W4)	Continuous processing	Streaming checkpoint in shared storage
Manufacturing	Predictive maintenance	ML (W2), streaming (W4)	Bounded lost training work	Adaptive stage checkpoints
Smart cities	Mobility and energy analytics	ETL (W1), graph (W3)	Long pipelines	Lineage cut by committed stages
Cybersecurity	Intrusion and log analytics	Streaming (W4), ML (W2)	Availability during incidents	Health-driven failover

G. LESSONS FOR ADOPTION

The use cases suggest three lessons for adopting the approach. First, the largest benefit is expected for long, expensive pipelines whose results are needed by a deadline, such as model retraining or end-of-day risk aggregation; short ad hoc queries gain little because restarting them is cheap. Second, domains with data-residency constraints need placement policies before automatic failover is enabled, otherwise recovery could create a compliance incident. Third, teams must adopt the discipline of deterministic, idempotent stages; this is good practice anyway, but it requires care with random seeds, timestamps and external side effects.

XIX. BUSINESS IMPACT ASSESSMENT

The business impact of RMC-Spark can be estimated with a simple model that separates what is measured from what is organisation-specific. Let N_f be the number of failures per year that affect analytics jobs, D_{base} and D_{rmc} the mean downtime per failure without and with RMC-Spark, C_d the cost of one hour of analytics downtime, and C_{extra} the yearly cost of the additional capacity and operations that the second cluster and the controller require. The net annual benefit is

$$\Delta B = N_f(D_{\text{base}} - D_{\text{rmc}})C_d - C_{\text{extra}} \qquad (12)$$

D_{base} and D_{rmc} come from the experiments (Table 11); N_f, C_d and C_{extra} must be supplied by the adopting organisation, because they vary by orders of magnitude between, for example, a bank's fraud pipeline and a monthly reporting job. Table 15 structures the assessment, and Fig. 28 presents it as a dashboard.



FIGURE 28. Business impact dashboard: downtime avoided, recomputation avoided and net benefit from Eq. (12) as functions of the organisation's failure rate and downtime cost. [[Generate after the experiments by inserting the measured D_{base} and D_{rmc}.]]

TABLE 15
COST-BENEFIT ANALYSIS

Item	Symbol	Source	Value
Failures per year affecting analytics	N_f	Organisation's incident history	[[organisation-specific]]
Mean downtime per failure, baseline	D_{base}	Experiments (B1 or B3)	[[measured]]
Mean downtime per failure, RMC-Spark	D_{rmc}	Experiments (P)	[[measured]]
Cost of one hour of analytics downtime	C_d	Business owner	[[organisation-specific]]
Extra capacity and operations per year	C_{extra}	Platform team	[[organisation-specific]]
Recomputation avoided per failure	ΔW	Experiments (stage events)	[[measured]]
Fault-free overhead	O	Experiments, Eq. (11)	[[measured]]
Net annual benefit	ΔB	Eq. (12)	[[computed]]

Three effects contribute to the benefit beyond the downtime term. Recomputation avoided frees capacity, because work that is restored instead of recomputed does not consume cluster time. Operator time is saved, because failovers no longer require someone to diagnose the outage and resubmit jobs, and the decision log shortens post-incident analysis. Finally, the second cluster is not idle capacity: under normal operation, the placement cost spreads jobs over both clusters, so the extra cost C_{extra} buys throughput as well as resilience. The main cost that offsets these benefits is the fault-free overhead O, which every job pays. [[Report the measured O and conclude whether, for a plausible range of N_f and C_d, the benefit is positive.]] Studies of SLA management in cloud computing show that availability commitments are usually tied to financial penalties [38], which gives organisations a concrete basis for C_d.

A. SENSITIVITY OF THE BUSINESS CASE

The business case is most sensitive to the product of failure frequency and downtime cost. For organisations with rare cluster-level failures and low downtime cost, the extra capacity may not pay off on resilience grounds alone, and the approach is attractive mainly when the second cluster is needed for capacity anyway. For organisations whose

analytics feed time-critical decisions, even one avoided outage per year can justify the investment. [[Insert a sensitivity table or plot of the net benefit over a range of failure rates and downtime costs using the measured D_{base} and D_{rmc}.]]

XX. CHALLENGES AND LIMITATIONS

A. CROSS-CLUSTER SYNCHRONISATION

RMC-Spark synchronises clusters only through shared object storage and a central controller. This keeps the design simple, but it makes both the object store and the controller critical. In the testbed, the single MinIO instance and the single controller container are single points of failure; a production deployment requires replicated object storage across failure domains and several controller replicas with leader election, so that exactly one replica acts on each job. Fig. 29 places these and the following limitations on a roadmap.

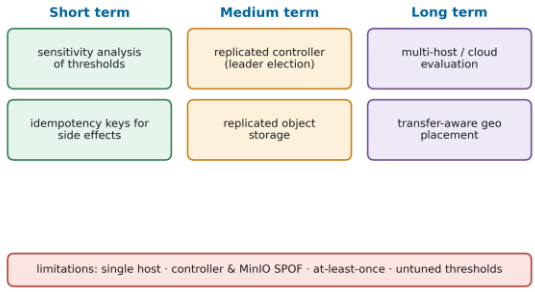


FIGURE 29. Challenges and limitations roadmap: current limitations of the testbed and design, the risk each one poses, and the mitigation planned in the short, medium and long term.

B. EXECUTION SEMANTICS

Stage execution is at least once. If a migrated attempt overlaps with a late write of the old attempt, both write the same deterministic output and the commit marker is rewritten, so the result is correct but work is duplicated. Stages with external side effects, such as sending notifications or writing to a database, would need deduplication or idempotency keys. Determinism is also a requirement on the user: stages with unseeded randomness would produce different results after resumption.

C. LATENCY OVERHEAD AND NETWORK BOTTLENECKS

Checkpoints cost time and storage bandwidth, and materialising each stage to measure its compute time adds a pass over the data. In geo-distributed deployments, reading committed stages from a remote store adds wide-area latency and cost, which geo-distributed analytics research shows can dominate job time [89][90]. The transfer term of the placement cost is the hook for handling this, but it is not exercised in the single-host testbed.

D. OPERATIONAL COMPLEXITY AND COST

Running two clusters and a controller is more complex than running one cluster, and the thresholds of the health model must be understood by the operators who rely on them. The thresholds were set by reasoning about the signals rather than tuned on data, and a sensitivity analysis is still missing. Deployment cost rises with the second cluster, although, as Section XIX notes, the second cluster also adds throughput.

E. DEPENDENCE ON DETERMINISM AND DATA SIZE

The approach assumes deterministic stages and moderate stage outputs. Stages that call external services, use unseeded randomness or depend on wall-clock time violate the first assumption. Stages whose outputs are very large make checkpoints expensive; the adaptive policy then checkpoints rarely, and recovery approaches lineage recomputation. Very small stages, conversely, add materialisation overhead without much benefit, and a production library would merge them.

F. THREATS TO VALIDITY

Internal validity is limited by the shared host: clusters compete for CPU, memory and disk, so a fault on one cluster can affect the other, and absolute timings do not transfer to other machines. External validity is limited by the workloads and data sizes, which are small compared with production analytics, and by emulated rather than real network faults. Construct validity depends on the metric definitions; using recovery criteria from published benchmarks [53] mitigates this risk. Conclusion validity is limited by three repetitions per cell, which is why comparisons are made over all faults of a configuration with non-parametric tests.

XXI. FUTURE RESEARCH DIRECTIONS

Fig. 30 outlines a research roadmap. The first direction is predictive recovery. The health model is reactive, but it has a simple interface: any model that outputs an anomaly level per cluster can contribute a signal. Failure predictors trained on cluster traces [35][60] and workload forecasting models [64] could add a predicted-failure signal, allowing the controller to checkpoint or migrate before the failure, as proactive cloud schedulers already do [62].

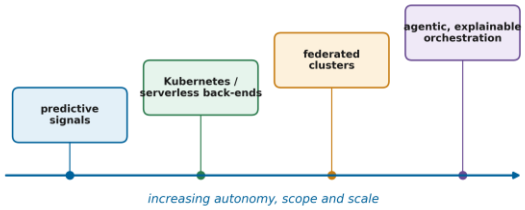


FIGURE 30. Future research roadmap: predictive recovery, federated multi-cluster management, Kubernetes and serverless back-ends, and agentic, explainable orchestration.

The second direction is federated cluster management across organisations and clouds. Federated learning shows how models can be trained across sites without moving raw data [126], and distributed machine learning systems face similar placement problems [127]. A federated RMC-Spark would place and move jobs across administrative domains under data-residency constraints, extending the jurisdiction-aware placement of Section XII.

The third direction is new execution back-ends. Kubernetes-hosted Spark and serverless analytics change how drivers and executors are started [15][16], and the controller's submit and stop

operations could target them without changing the decision logic. Evaluating a Kubernetes back-end is the most immediate extension of the experimental matrix.

The fourth direction is agentic and self-healing orchestration. Large language models are being explored for AIOps tasks such as incident analysis and remediation [67], and the joint view of MLOps and AIOps emphasises closing the loop between operations and model development [66]. An agent could read the decision log, propose configuration changes, or explain incidents in natural language, while the deterministic controller keeps responsibility for safety-critical actions. Fig. 31 sketches such a next-generation architecture.

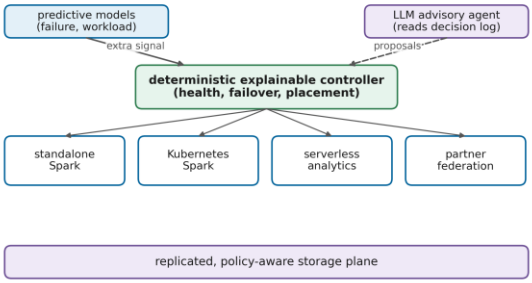


FIGURE 31. Next-generation resilient analytics architecture: predictive signals and an advisory agent added to the deterministic, explainable control loop, with federated clusters and multiple execution back-ends.

Further directions concern the controller itself. Learned checkpoint policies could replace the analytical interval when failures are not memoryless, for instance by conditioning on time of day or on recent anomaly scores, similar to convergence-aware checkpointing for training jobs [51]. Formal verification of the state machine and recovery rules would provide guarantees that testing cannot, for example that no job is ever migrated while making progress. Energy and cost awareness could be added to the placement cost, so that failover prefers clusters with spare capacity or lower price, which multi-cloud scheduling research already considers [25][84]. Finally, sensitivity analysis of thresholds and weights, and their automatic calibration from incident data, would reduce the dependence on expert judgement noted in Section XI.

XXII. CONCLUSION

This paper addressed a gap between the fault tolerance that Apache Spark provides inside a cluster and the availability that analytics consumers expect when a whole cluster fails. It presented RMC-Spark, which combines resumable data science program stages with a cross-cluster resilience controller. The rds library makes PySpark stages idempotent and resumable through a commit-marker protocol on shared object storage and checkpoints them adaptively using the Young and Daly optimum interval with an online MTBF estimate. The controller scores cluster health from five explainable signals, detects failures with a hysteresis state machine, migrates only jobs that stopped making progress, places jobs with an interpretable cost function, and records every decision with its cause. Fig. 32 summarises how these contributions relate to the problems identified in Section II.

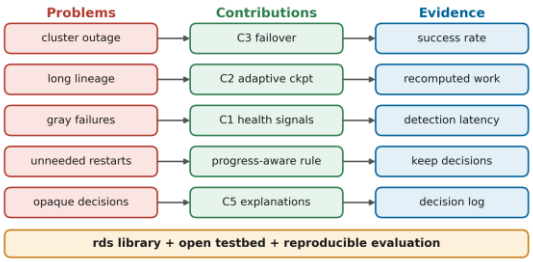


FIGURE 32. Research contribution framework: problems identified in Section II, the components that address them, and the evidence used to evaluate each one.

The implementation is complete and openly available, its decision logic is verified by automated tests including a simulated cluster outage, and a reproducible evaluation with nine fault types, four workloads and five configurations has been specified together with its statistical analysis. [[Replace this sentence with the two or three main measured findings and whether hypotheses H1 to H5 were supported.]] For practitioners, the main implication is that cross-cluster resilience for analytics does not require changes to the engine: a thin programming library and an explainable controller are sufficient to turn cluster outages from failed jobs into automatic recoveries, at a cost that each organisation can weigh with the model of Section XIX. Future work will add predictive signals, Kubernetes and federated back-ends, and replicated control and storage planes, and will evaluate the approach at production scale.

Several lessons emerged from designing and implementing the system. Placing drivers outside the clusters was the single most important decision, because it made the difference between a job that dies with its cluster and a job that can be moved. Distinguishing degradation from failure, and failure from a stopped job, avoided the main weakness of timeout-based high availability, namely restarting work that would have finished. Making every decision explainable required no sophisticated machinery once the models were chosen to be additive, which suggests that explainability is often a design choice rather than an afterthought. Finally, expressing the experiments as code, with seeded data and scripted faults, made the evaluation reproducible by anyone with a workstation and Docker, which is valuable for teaching as well as for research.

DATA AVAILABILITY

The synthetic datasets analysed in this study are generated deterministically by the scripts in the project repository [[GitHub link]]. The public datasets are available from the NYC Taxi and Limousine Commission (TLC Trip Record Data), the UCI Machine Learning Repository (AI4I 2020 Predictive Maintenance Dataset, identifier 601) and the Machine Learning Group of ULB (Credit Card Fraud Detection). The raw experiment logs and measurements generated during the current study are available in the same repository under results/.

APPENDIX

A. SPARK AND CONTROLLER CONFIGURATION

The complete configuration is part of the repository: docker-compose.yml defines the testbed, docker/spark-defaults.conf the Spark settings, configs/clusters.yaml all controller thresholds and weights, and experiments/configs/full.yaml the experiment matrix. The values used in this paper are those of Tables 6, 7 and 9.

B. REPRODUCTION STEPS

The experiments can be reproduced with the following steps, which are described in detail in the repository README: build the image and start the testbed with Docker Compose; generate the datasets with the data generator; run the smoke experiment to validate the setup; calibrate the fault injection times to the clean runtimes of the reproducing machine; run the full matrix; and generate all tables and figures with the analysis script. The 56 automated tests run without Docker or Spark and verify the policy, detection, placement, reliability and recovery logic.

C. RECOVERY RULES

The recovery rules applied by the controller in each recovery mode are listed in Table 16.

TABLE 16
RECOVERY RULES PER MODE

Situation	none (B1, B2)	naive (B3)	smart (RMC-Spark)
Driver exits with error, cluster healthy	Job fails	Rerun on passive cluster	Retry on same cluster, then fail over
Driver exits with error, cluster suspect or failed	Job fails	Rerun on passive cluster	Fail over to best cluster
Cluster failed, job still progressing	Job continues	Job restarted elsewhere	Job kept, decision logged
Cluster failed, job stalled 10 s or more	Job hangs until deadline	Restarted after 30 s timeout	Migrated to argmin J
No eligible cluster	Not applicable	Job waits	Job waits, then resumes

REFERENCES

[1] H. Zhou, G. Sun, S. Fu, L. Wang, J. Hu, and Y. Gao, "Internet Financial Fraud Detection Based on a Distributed Big Data Approach With Node2vec," *IEEE Access*, vol. 9, pp. 43378–43386, 2021, doi: 10.1109/access.2021.3062467.

[2] R. Rosati et al., "From knowledge-based to big data analytic model: a novel IoT and machine learning based decision support system for predictive maintenance in Industry 4.0," *Journal of Intelligent Manufacturing*, vol. 34, no. 1, pp. 107–121, 2022, doi: 10.1007/s10845-022-01960-x.

[3] M. Zaharia et al., "Apache Spark," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016, doi: 10.1145/2934664.

[4] S. Tang, B. He, C. Yu, Y. Li, and K. Li, "A Survey on Spark Ecosystem: Big Data Processing Infrastructure, Machine Learning, and Applications," *IEEE Transactions on Knowledge and Data Engineering*, p. 1, 2020, doi: 10.1109/tkde.2020.2975652.

[5] M. Zaharia et al., "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. 9th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, pp. 15–28, 2012.

[6] S. Isukapalli and S. N. Srirama, "A systematic survey on fault-tolerant solutions for distributed data analytics: Taxonomy, comparison, and future directions," *Computer Science Review*, vol. 53, Art. no. 100660, 2024, doi: 10.1016/j.cosrev.2024.100660.

[7] J. Dean and S. Ghemawat, "MapReduce," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008, doi: 10.1145/1327452.1327492.

[8] V. K. Vavilapalli et al., "Apache Hadoop YARN," in *Symposium on Cloud Computing*, pp. 1–16, 2013, doi: 10.1145/2523616.2523633.

[9] M. Armbrust et al., "Spark SQL," in *Proceedings - ACM-SIGMOD International Conference on Management of Data*, pp. 1383–1394, 2015, doi: 10.1145/2723372.2742797.

[10] M. Armbrust et al., "Structured Streaming," in *Proceedings - ACM-SIGMOD International Conference on Management of Data*, pp. 601–613, 2018, doi: 10.1145/3183713.3190664.

[11] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, "Apache Flink: Stream and batch processing in a single engine," *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, vol. 36, no. 4, pp. 28–38, 2015.

[12] N. Ahmed, A. L. C. Barczak, T. Sušnjak, and M. A. Rashid, "A comprehensive performance analysis of Apache Hadoop and Apache Spark for large scale data sets using HiBench," *Journal Of Big Data*, vol. 7, no. 1, 2020, doi: 10.1186/s40537-020-00388-5.

[13] A. Verma, L. D. Pedrosa, M. R. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *European Conference on Computer Systems*, pp. 1–17, 2015, doi: 10.1145/2741948.2741964.

[14] B. Burns, B. Grant, D. Oppenheimer, E. A. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016, doi: 10.1145/2890784.

[15] J. Schleier-Smith et al., "What serverless computing is and should become," *Communications of the ACM*, vol. 64, no. 5, pp. 76–84, 2021, doi: 10.1145/3406011.

[16] H. B. Hassan, S. Barakat, and Q. I. Sarhan, "Survey on serverless computing," *Journal of Cloud Computing Advances Systems and Applications*, vol. 10, no. 1, 2021, doi: 10.1186/s13677-021-00253-7.

[17] R. Hai, C. Koutras, C. Quix, and M. Jarke, "Data Lakes: A Survey of Functions and Systems," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12571–12590, 2023, doi: 10.1109/tkde.2023.3270101.

[18] A. A. Harby and F. H. Zulkernine, "Data Lakehouse: A survey and experimental study," *Information Systems*, vol. 127, Art. no. 102460, 2024, doi: 10.1016/j.is.2024.102460.

[19] M. Fragkoulis, P. Carbone, V. Kalavri, and A. Katsifodimos, "A survey on the evolution of stream processing systems," *The VLDB Journal*, vol. 33, no. 2, pp. 507–541, 2023, doi: 10.1007/s00778-023-00819-8.

[20] E. Mehmood and T. Anees, "Challenges and Solutions for Processing Real-Time Big Data Stream: A Systematic Literature Review," *IEEE Access*, vol. 8, pp. 119123–119143, 2020, doi: 10.1109/access.2020.3005268.

[21] A. Almeida, S. Brás, S. Sargento, and F. C. Pinto, "Time series big data: a survey on data stream frameworks, analysis and algorithms," *Journal Of Big Data*, vol. 10, no. 1, p. 83, 2023, doi: 10.1186/s40537-023-00760-1.

[22] A. U. Rehman, R. L. Aguiar, and J. P. Barraca, "Fault-Tolerance in the Scope of Cloud Computing," *IEEE Access*, vol. 10, pp. 63422–63441, 2022, doi: 10.1109/access.2022.3182211.

[23] M. Kweun, W. Lee, G. Kim, J. Hwang, and Y. Lee, "Lineage Checkpoint Approach for Long-lineage Problem in Apache Spark," in *IEEE International Conference on Big Data (BigData)*, pp. 5733–5735, 2020, doi: 10.1109/bigdata50022.2020.9377896.

[24] P. Huang et al., "Gray Failure," in *Workshop on Hot Topics in Operating Systems*, pp. 150–155, 2017, doi: 10.1145/3102980.3103005.

[25] O. Tomarchio, D. Calcaterra, and G. Modica, "Cloud resource orchestration in the multi-cloud landscape: a systematic review of existing frameworks," *Journal of Cloud Computing Advances Systems and Applications*, vol. 9, no. 1, 2020, doi: 10.1186/s13677-020-00194-7.

[26] F. M. Awaysheh, M. Alazab, S. Garg, D. T. Niyato, and C. V. Verikoukis, "Big Data Resource Management & Networks: Taxonomy, Survey, and Future Directions," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2098–2130, 2021, doi: 10.1109/comst.2021.3094993.

[27] Z. Wang and A. Aiken, "Efficient Fault Tolerance for Pipelined Query Engines via Write-ahead Lineage," in *Proceedings - International Conference on Data Engineering*, pp. 436–448, 2024, doi: 10.1109/icde60146.2024.00040.

[28] G. Raj, K. Kaur, A. P. Agrawal, P. Kumar, P. K. Pateriya, and A. Punhani, "A meta Proximal policy optimization driven task interaction graph scheduler with predictive resource provisioning and fault-tolerant micro-checkpointing for adaptive spark workflow scheduling in multi-cloud systems," *Engineering Applications of Artificial Intelligence*, vol. 182, Art. no. 115968, 2026, doi: 10.1016/j.engappai.2026.115968.

[29] S. Nastić et al., "SLOC: Service Level Objectives for Next Generation Cloud Computing," *IEEE Internet Computing*, vol. 24, no. 3, pp. 39–50, 2020, doi: 10.1109/mic.2020.2987739.

[30] Y. Li et al., "Predicting Node Failures in an Ultra-Large-Scale Cloud Computing Platform," *ACM Transactions on Software Engineering and Methodology*, vol. 29, no. 2, pp. 1–24, 2020, doi: 10.1145/3385187.

[31] Q. Lin et al., "Predicting Node failure in cloud service systems," in *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pp. 480–490, 2018, doi: 10.1145/3236024.3236060.

[32] J. Zhang et al., "Minority Disk Failure Prediction Based on Transfer Learning in Large Data Centers of Heterogeneous Disk Systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 9, pp. 2155–2169, 2020, doi: 10.1109/tpds.2020.2985346.

[33] S. Han, P. P. C. Lee, Z. Shen, C. He, Y. Liu, and T. Huang, "StreamDFP: A General Stream Mining Framework for Adaptive Disk Failure Prediction," *IEEE Transactions on Computers*, vol. 72, no. 2, pp. 520–534, 2022, doi: 10.1109/tc.2022.3160365.

[34] K. L. Bawankule, R. K. Dewang, and A. K. Singh, "A classification framework for straggler mitigation and management in a heterogeneous Hadoop cluster: A state-of-art survey," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 9, pp. 7621–7644, 2022, doi: 10.1016/j.jksuci.2022.02.021.

[35] J. Gao, H. Wang, and H. Shen, "Task Failure Prediction in Cloud Data Centers Using Deep Learning," *IEEE Transactions on Services Computing*, vol. 15, no. 3, pp. 1411–1422, 2020, doi: 10.1109/tsc.2020.2993728.

[36] M. A. Shahid, N. Islam, M. M. Alam, M. S. Mazliham, and S. Musa, "Towards Resilient Method: An exhaustive survey of fault tolerance methods in the cloud computing environment," *Computer Science Review*, vol. 40, Art. no. 100398, 2021, doi: 10.1016/j.cosrev.2021.100398.

[37] M. Kirti, A. K. Maurya, and R. S. Yadav, "Fault-tolerance approaches for distributed and cloud computing environments: A systematic review, taxonomy and future directions," *Concurrency and Computation Practice and Experience*, vol. 36, no. 13, 2024, doi: 10.1002/cpe.8081.

[38] F. Qazi, D. Kwak, F. G. Khan, F. Ali, and S. U. Khan, "Service Level Agreement in cloud computing: Taxonomy, prospects, and challenges," *Internet of Things*, vol. 25, Art. no. 101126, 2024, doi: 10.1016/j.iot.2024.101126.

[39] F. Dinu and T. S. E. Ng, "RCMP: Enabling Efficient Recomputation Based Failure Resilience for Big Data Analytics," in *Proceedings - IEEE International Parallel and Distributed Processing Symposium*, pp. 962–971, 2014, doi: 10.1109/ipdps.2014.102.

[40] C. Li, Q. Cai, and Y. Luo, "Data balancing-based intermediate data partitioning and check point-based cache recovery in Spark environment," *The Journal of Supercomputing*, vol. 78, no. 3, pp. 3561–3604, 2021, doi: 10.1007/s11227-021-04000-2.

[41] J. Shang, Y. Zhang, J. Wang, Z. Wu, and Z. Xiao, "Distributed cache strategy based on LT codes under spark platform," *The Journal of Supercomputing*, vol. 80, no. 11, pp. 16519–16545, 2024, doi: 10.1007/s11227-024-06095-9.

[42] M. He, Z. Fu, and W. Tian, "Optimization of fault tolerance for iterative graph algorithm in spark GraphX based on high performance computing cluster," *CCF Transactions on High Performance Computing*, vol. 7, no. 5, pp. 465–477, 2025, doi: 10.1007/s42514-025-00225-2.

[43] J. W. Young, "A first order approximation to the optimum checkpoint interval," *Communications of the ACM*, vol. 17, no. 9, pp. 530–531, 1974, doi: 10.1145/361147.361115.

[44] J. T. Daly, "A higher order estimate of the optimum checkpoint interval for restart dumps," *Future Generation Computer Systems*, vol. 22, no. 3, pp. 303–312, 2006, doi: 10.1016/j.future.2004.11.016.

[45] P. Sigdel, X. Yuan, and N. Tzeng, "Realizing Best Checkpointing Control in Computing Systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 2, pp. 315–329, 2020, doi: 10.1109/tpds.2020.3015805.

[46] S. Jayasekara, A. Harwood, and S. A. Karunasekera, "A utilization model for optimization of checkpoint intervals in distributed stream processing systems," *Future Generation Computer Systems*, vol. 110, pp. 68–79, 2020, doi: 10.1016/j.future.2020.04.019.

[47] S. Jayasekara, S. A. Karunasekera, and A. Harwood, "Optimizing checkpoint-based fault-tolerance in distributed stream processing systems: Theory to practice," *Software Practice and Experience*, vol. 52, no. 1, pp. 296–315, 2021, doi: 10.1002/spe.3021.

[48] Z. Zhang, W. Li, X. Qing, X. Liu, and H. Liu, "Research on Optimal Checkpointing-Interval for Flink Stream Processing Applications," *Mobile Networks and Applications*, vol. 26, no. 5, pp. 1950–1959, 2021, doi: 10.1007/s11036-020-01729-7.

[49] S. M. A. Akber, H. Chen, and H. Jin, "FATM: A failure-aware adaptive fault tolerance model for distributed stream processing systems," *Concurrency and Computation Practice and Experience*, vol. 33, no. 10, 2021, doi: 10.1002/cpe.6167.

[50] C. Lin, L. Wang, and S. Chang, "Incremental Checkpointing for Fault-Tolerant Stream Processing Systems: A Data Structure Approach," *IEEE Transactions on Emerging Topics in Computing*, vol. 10, no. 1, pp. 124–136, 2020, doi: 10.1109/tetc.2020.2986487.

[51] H. Li, Z. Wang, H. Zhao, M. Zhang, X. Li, and H. Xu, "Convergence-aware optimal checkpointing for exploratory deep learning training jobs," *Future Generation Computer Systems*, vol. 164, Art. no. 107597, 2024, doi: 10.1016/j.future.2024.107597.

[52] G. v. Dongen and D. V. d. Poel, "A Performance Analysis of Fault Recovery in Stream Processing Frameworks," *IEEE Access*, vol. 9, pp. 93745–93763, 2021, doi: 10.1109/access.2021.3093208.

[53] A. Vogel, S. Henning, E. Pérez-Wohlfeil, O. Ertl, and R. Rabiser, "A Comprehensive Benchmarking Analysis of Fault Recovery in Stream Processing Frameworks," in *ACM International Conference on Distributed and Event-Based Systems (DEBS)*, pp. 171–182, 2024, doi: 10.1145/3629104.3666040.

[54] J. Tahir, R. Mayer, C. Doblander, and H. Jacobsen, "How Reliable are Streams? End-to-End Processing-Guarantee Validation and Performance Benchmarking of Stream Processing Systems," *Proceedings of the VLDB Endowment*, vol. 18, no. 3, pp. 585–598, 2024, doi: 10.14778/3712221.3712227.

[55] M. Zaharia, A. Konwinski, A. D. Joseph, R. Katz, and I. Stoica, "Improving MapReduce performance in heterogeneous environments," in *Proc. 8th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, pp. 29–42, 2008.

[56] G. Ananthanarayanan et al., "Reining in the outliers in map-reduce clusters using Mantri," in *Proc. 9th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, pp. 265–278, 2010.

[57] H. Du and S. Zhang, "Hawkeye: Adaptive Straggler Identification on Heterogeneous Spark Cluster With Reinforcement Learning," *IEEE Access*, vol. 8, pp. 57822–57832, 2020, doi: 10.1109/access.2020.2982320.

[58] M. Farhang and F. Safi-Esfahani, "Recognizing MapReduce Straggler Tasks in Big Data Infrastructures Using Artificial Neural Networks," *Journal of Grid Computing*, vol. 18, no. 4, pp. 879–901, 2020, doi: 10.1007/s10723-020-09514-2.

[59] W. Li, D. Liu, K. Chen, K. Li, and H. Qi, "Hone: Mitigating Stragglers in Distributed Stream Processing With Tuple Scheduling," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 8, pp. 2021–2034, 2021, doi: 10.1109/tpds.2021.3051059.

[60] T. N. T. Asmawi, A. B. Ismail, and J. Shen, "Cloud failure prediction based on traditional machine learning and deep learning," *Journal of Cloud Computing Advances Systems and Applications*, vol. 11, no. 1, 2022, doi: 10.1186/s13677-022-00327-0.

[61] H. Bommala, V. U. Maheswari, R. Aluvalu, and S. Mudrakola, "Machine learning job failure analysis and prediction model for the cloud environment," *High-Confidence Computing*, vol. 3, no. 4, Art. no. 100165, 2023, doi: 10.1016/j.hcc.2023.100165.

[62] Y. Alahmad, T. Daradkeh, and A. Agarwal, "Proactive Failure-Aware Task Scheduling Framework for Cloud Computing," *IEEE Access*, vol. 9, pp. 106152–106168, 2021, doi: 10.1109/access.2021.3101147.

[63] D. Saxena and A. K. Singh, "OFP-TM: an online VM failure prediction and tolerance model towards high availability of cloud computing environments," *The Journal of Supercomputing*, vol. 78, no. 6, pp. 8003–8024, 2022, doi: 10.1007/s11227-021-04235-z.

[64] D. Saxena, J. Kumar, A. K. Singh, and S. Schmid, "Performance Analysis of Machine Learning Centered Workload Prediction Models for Cloud," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 4, pp. 1313–1330, 2023, doi: 10.1109/tpds.2023.3240567.

[65] P. Notaro, J. Cardoso, and M. Gerndt, "A Survey of AIOps Methods for Failure Management," *ACM Transactions on Intelligent Systems and Technology*, vol. 12, no. 6, pp. 1–45, 2021, doi: 10.1145/3483424.

[66] J. Díaz-de-Arcaya, A. I. Torre-Bastida, G. Zárate, R. Miñón, and A. Almeida, "A Joint Study of the Challenges, Opportunities, and Roadmap of MLOps and AIOps: A Systematic Survey," *ACM Computing Surveys*, vol. 56, no. 4, pp. 1–30, 2023, doi: 10.1145/3625289.

[67] L. Zhang et al., "A Survey of AIOps in the Era of Large Language Models," *ACM Computing Surveys*, vol. 58, no. 2, pp. 1–35, 2025, doi: 10.1145/3746635.

[68] Z. Z. Darban, G. I. Webb, S. Pan, C. C. Aggarwal, and M. Salehi, "Deep Learning for Time Series Anomaly Detection: A Survey," *ACM Computing Surveys*, vol. 57, no. 1, pp. 1–42, 2024, doi: 10.1145/3691338.

[69] K. Choi, J. Yi, C. Park, and S. Yoon, "Deep Learning for Anomaly Detection in Time-Series Data: Review, Analysis, and Guidelines," *IEEE Access*, vol. 9, pp. 120043–120065, 2021, doi: 10.1109/access.2021.3107975.

[70] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, “Deep learning for anomaly detection in log data: A survey,” *Machine Learning with Applications*, vol. 12, Art. no. 100470, 2023, doi: 10.1016/j.mlwa.2023.100470.

[71] D. A. Bhanage, A. Pawar, and K. V. Kotecha, “IT Infrastructure Anomaly Detection and Failure Handling: A Systematic Literature Review Focusing on Datasets, Log Preprocessing, Machine & Deep Learning Approaches and Automated Tool,” *IEEE Access*, vol. 9, pp. 156392–156421, 2021, doi: 10.1109/access.2021.3128283.

[72] A. Hrusto, N. b. Ali, E. Engström, and Y. Wang, “Monitoring Data for Anomaly Detection in Cloud-Based Systems: A Systematic Mapping Study,” *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 4, pp. 1–36, 2025, doi: 10.1145/3744556.

[73] J. G. Almaraz-Rivera, “An Anomaly-based Detection System for Monitoring Kubernetes Infrastructures,” *IEEE Latin America Transactions*, vol. 21, no. 3, pp. 457–465, 2023, doi: 10.1109/tla.2023.10068850.

[74] M. Ma, W. Lin, D. Pan, and P. Wang, “ServiceRank: Root Cause Identification of Anomaly in Large-Scale Microservice Architectures,” *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 3087–3100, 2021, doi: 10.1109/tdsc.2021.3083671.

[75] J. Soldani, S. Forti, L. Roveroni, and A. Brogi, “Explaining Microservices' Cascading Failures From Their Logs,” *Software Practice and Experience*, vol. 55, no. 5, pp. 809–828, 2024, doi: 10.1002/spe.3400.

[76] J. Soldani and A. Brogi, “Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey,” *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, 2022, doi: 10.1145/3501297.

[77] N. Fu, G. Cheng, Y. Teng, G. Dai, S. Yu, and Z. Chen, “Intelligent Root Cause Localization in MicroService Systems: A Survey and New Perspectives,” *ACM Computing Surveys*, vol. 57, no. 12, pp. 1–37, 2025, doi: 10.1145/3736755.

[78] M. C. Carrión, “Kubernetes Scheduling: Taxonomy, Ongoing Issues and Challenges,” *ACM Computing Surveys*, vol. 55, no. 7, pp. 1–37, 2022, doi: 10.1145/3539606.

[79] K. A. K. Senjab, S. Abbas, N. Ahmed, and A. u. R. Khan, “A survey of Kubernetes scheduling algorithms,” *Journal of Cloud Computing Advances Systems and Applications*, vol. 12, no. 1, 2023, doi: 10.1186/s13677-023-00471-1.

[80] Z. Rejiba and J. Chamanara, “Custom Scheduling in Kubernetes: A Survey on Common Problems and Solution Approaches,” *ACM Computing Surveys*, vol. 55, no. 7, pp. 1–37, 2022, doi: 10.1145/3544788.

[81] Z. Zhong and R. Buyya, “A Cost-Efficient Container Orchestration Strategy in Kubernetes-Based Cloud Computing Infrastructures with Heterogeneous Resources,” *ACM Transactions on Internet Technology*, vol. 20, no. 2, pp. 1–24, 2020, doi: 10.1145/3378447.

[82] S. Tuli, S. R. Poojara, S. N. Srirama, G. Casale, and N. R. Jennings, “COSCO: Container Orchestration Using Co-Simulation and Gradient Based Optimization for Fog Computing Environments,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 1, pp. 101–116, 2021, doi: 10.1109/tpds.2021.3087349.

[83] N. Zhou et al., “Container orchestration on HPC systems through Kubernetes,” *Journal of Cloud Computing Advances Systems and Applications*, vol. 10, no. 1, 2021, doi: 10.1186/s13677-021-00231-z.

[84] Q. Zhu, H. Tang, J. Huang, and Y. Hou, “Task Scheduling for Multi-Cloud Computing Subject to Security and Reliability Constraints,” *IEEE/CAA Journal of Automatica Sinica*, vol. 8, no. 4, pp. 848–865, 2021, doi: 10.1109/jas.2021.1003934.

[85] Q. Pu et al., “Low Latency Geo-distributed Data Analytics,” , pp. 421–434, 2015, doi: 10.1145/2785956.2787505.

[86] T. Z. Emara and J. Z. Huang, “Distributed Data Strategies to Support Large-Scale Data Analysis Across Geo-Distributed Data Centers,” *IEEE Access*, vol. 8, pp. 178526–178538, 2020, doi: 10.1109/access.2020.3027675.

[87] L. Chen, S. Liu, and B. Li, “Optimizing Network Transfers for Data Analytic Jobs Across Geo-Distributed Datacenters,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 2, pp. 403–414, 2021, doi: 10.1109/tpds.2021.3093232.

[88] K. Oh, M. Zhang, A. Chandra, and J. Weissman, “Network Cost-Aware Geo-Distributed Data Analytics System,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 6, pp. 1407–1420, 2021, doi: 10.1109/tpds.2021.3108893.

[89] H. Mostafaei, G. Smaragdakis, T. Zinner, and A. Feldmann, “Delay-Resistant Geo-Distributed Analytics,” *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 4734–4749, 2022, doi: 10.1109/tnsm.2022.3192710.

[90] M. Bergui, S. Najah, and N. S. Nikolov, “A survey on bandwidth-aware geo-distributed frameworks for big-data analytics,” *Journal Of Big Data*, vol. 8, no. 1, 2021, doi: 10.1186/s40537-021-00427-9.

[91] B. K. Ray, A. Saha, S. Khatua, and S. Roy, “Proactive Fault-Tolerance Technique to Enhance Reliability of Cloud Service in Cloud Federation Environment,” *IEEE Transactions on Cloud Computing*, vol. 10, no. 2, pp. 957–971, 2020, doi: 10.1109/tcc.2020.2968522.

[92] Y. Tian, J. Tian, and N. Li, “Cloud reliability and efficiency improvement via failure risk based proactive actions,” *Journal of Systems and Software*, vol. 163, Art. no. 110524, 2020, doi: 10.1016/j.jss.2020.110524.

[93] S. Meng, L. Luo, X. Qiu, and Y. Dai, “Service-Oriented Reliability Modeling and Autonomous Optimization of Reliability for Public Cloud Computing Systems,” *IEEE Transactions on Reliability*, vol. 71, no. 2, pp. 527–538, 2022, doi: 10.1109/tr.2022.3154651.

[94] P. Maciel et al., “A survey on reliability and availability modeling of edge, fog, and cloud computing,” *Journal of Reliable Intelligent Environments*, vol. 8, no. 3, pp. 227–245, 2021, doi: 10.1007/s40860-021-00154-1.

[95] A. Basiri et al., “Chaos Engineering,” *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016, doi: 10.1109/ms.2016.60.

[96] K. A. Torkura, M. I. H. Sukmana, F. Cheng, and C. Meinel, “CloudStrike: Chaos Engineering for Security and Resiliency in Cloud Infrastructure,” *IEEE Access*, vol. 8, pp.

123044–123060, 2020, doi: 10.1109/access.2020.3007338.

[97] J. Simonsson, L. Zhang, B. Morin, B. Baudry, and M. Monperrus, "Observability and chaos engineering on system calls for containerized applications in Docker," *Future Generation Computer Systems*, vol. 122, pp. 117–129, 2021, doi: 10.1016/j.future.2021.04.001.

[98] M. Fogli, C. Giannelli, F. Poltronieri, C. Stefanelli, and M. Tortonesi, "Chaos Engineering for Resilience Assessment of Digital Twins," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 2, pp. 1134–1143, 2023, doi: 10.1109/tii.2023.3264101.

[99] J. Owotogbe, I. Kumara, W. v. d. Heuvel, and D. A. Tamburri, "Chaos Engineering: A Multi-Vocal Literature Review," *ACM Computing Surveys*, vol. 58, no. 7, pp. 1–44, 2025, doi: 10.1145/3777375.

[100] H. Herodotou, Y. Chen, and J. Lu, "A Survey on Automatic Parameter Tuning for Big Data Processing Systems," *ACM Computing Surveys*, vol. 53, no. 2, pp. 1–37, 2020, doi: 10.1145/3381027.

[101] D. Buchaca, F. A. Portella, C. H. Ã. Costa, and J. L. Berral, "You Only Run Once: Spark Auto-Tuning From a Single Run," *IEEE Transactions on Network and Service Management*, vol. 17, no. 4, pp. 2039–2051, 2020, doi: 10.1109/tnsm.2020.3034824.

[102] Y. Li et al., "Towards General and Efficient Online Tuning for Spark," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3570–3583, 2023, doi: 10.14778/3611540.3611548.

[103] G. Cheng, S. Ying, and B. Wang, "Tuning configuration of apache spark on public clouds by combining multi-objective optimization and performance prediction model," *Journal of Systems and Software*, vol. 180, Art. no. 111028, 2021, doi: 10.1016/j.jss.2021.111028.

[104] G. Cheng, S. Ying, B. Wang, and Y. Li, "Efficient Performance Prediction for Apache Spark," *Journal of Parallel and Distributed Computing*, vol. 149, pp. 40–51, 2020, doi: 10.1016/j.jpdc.2020.10.010.

[105] S. Shah, Y. Amannejad, and D. Krishnamurthy, "Diaspore: Diagnosing Performance Interference in Apache Spark," *IEEE Access*, vol. 9, pp. 103230–103243, 2021, doi: 10.1109/access.2021.3098426.

[106] M. Dugré, V. Hayot-Sasson, and T. Glatard, "Performance comparison of Dask and Apache Spark on HPC systems for neuroimaging," *Concurrency and Computation Practice and Experience*, vol. 35, no. 21, 2023, doi: 10.1002/cpe.7635.

[107] P. Sawadogo and J. Darmont, "On data lake architectures and metadata management," *Journal of Intelligent Information Systems*, vol. 56, no. 1, pp. 97–120, 2020, doi: 10.1007/s10844-020-00608-7.

[108] S. A. Errami, H. Hajji, K. A. E. Kadi, and H. Badir, "Spatial big data architecture: From Data Warehouses and Data Lakes to the LakeHouse," *Journal of Parallel and Distributed Computing*, vol. 176, pp. 70–79, 2023, doi: 10.1016/j.jpdc.2023.02.007.

[109] H. Foidl, V. Golendukhina, R. Ramler, and M. Felderer, "Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers," *Journal of Systems and Software*, vol. 207, Art. no. 111855, 2023, doi: 10.1016/j.jss.2023.111855.

[110] I. Taleb, M. A. Serhani, C. Bouhaddioui, and R. Dssouli, "Big data quality framework: a holistic approach to continuous quality management," *Journal Of Big Data*, vol. 8, no. 1, 2021, doi: 10.1186/s40537-021-00468-0.

[111] A. H. Chillón, M. Klettke, D. S. Ruiz, and J. G. Molina, "A Generic Schema Evolution Approach for NoSQL and Relational Databases," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 2774–2789, 2024, doi: 10.1109/tkde.2024.3362273.

[112] M. Janssen, P. Brous, E. Estévez, L. S. Barbosa, and T. Janowski, "Data governance: Organizing data for trustworthy Artificial Intelligence," *Government Information Quarterly*, vol. 37, no. 3, Art. no. 101493, 2020, doi: 10.1016/j.giq.2020.101493.

[113] S. Nadal, P. Jovanović, B. Bilalli, and O. Romero, "Operationalizing and automating Data Governance," *Journal Of Big Data*, vol. 9, no. 1, p. 117, 2022, doi: 10.1186/s40537-022-00673-5.

[114] M. Usman, S. Ferlin, A. Brunström, and J. Taheri, "A Survey on Observability of Distributed Edge & Container-Based Microservices," *IEEE Access*, vol. 10, pp. 86904–86919, 2022, doi: 10.1109/access.2022.3193102.

[115] B. Li et al., "Enjoy your observability: an industrial survey of microservice tracing and analysis," *Empirical Software Engineering*, vol. 27, no. 1, p. 25, 2021, doi: 10.1007/s10664-021-10063-9.

[116] Y. Kang, L. Pan, and S. Liu, "Job scheduling for big data analytical applications in clouds: A taxonomy study," *Future Generation Computer Systems*, vol. 135, pp. 129–145, 2022, doi: 10.1016/j.future.2022.04.035.

[117] Y. K. Sanjalawe, S. Al-E'mari, S. Fraihat, and S. N. Makhadmeh, "AI-driven job scheduling in cloud computing: a comprehensive review," *Artificial Intelligence Review*, vol. 58, no. 7, 2025, doi: 10.1007/s10462-025-11208-8.

[118] H. E. Alhazmi, F. E. Eassa, and S. Sandokji, "Towards Big Data Security Framework by Leveraging Fragmentation and Blockchain Technology," *IEEE Access*, vol. 10, pp. 10768–10782, 2022, doi: 10.1109/access.2022.3144632.

[119] S. Lu, J. Zheng, Z. Cao, Y. Wang, and C. Gu, "A survey on cryptographic techniques for protecting big data security: present and forthcoming," *Science China Information Sciences*, vol. 65, no. 10, 2022, doi: 10.1007/s11432-021-3393-x.

[120] K. M. Batko and A. Ślęzak, "The use of Big Data Analytics in healthcare," *Journal Of Big Data*, vol. 9, no. 1, p. 3, 2022, doi: 10.1186/s40537-021-00553-4.

[121] M. Z. Kastouni and A. A. Lahcen, "Big data analytics in telecommunications: Governance, architecture and use cases," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 6, pp. 2758–2770, 2020, doi: 10.1016/j.jksuci.2020.11.024.

[122] M. Fugini, J. Finocchi, and P. Locatelli, "A Big Data Analytics Architecture for Smart Cities and Smart Companies," *Big Data Research*, vol. 24, Art. no. 100192, 2021, doi: 10.1016/j.bdr.2021.100192.

[123] H. Liang, Z. Zhang, C. Hu, Y. Gong, and D. Cheng, "A Survey on Spatio-Temporal Big

Data Analytics Ecosystem: Resource Management, Processing Platform, and Applications," *IEEE Transactions on Big Data*, vol. 10, no. 2, pp. 174–193, 2023, doi: 10.1109/tbdata.2023.3342619.

[124] F. Ullah and M. A. Babar, "On the scalability of Big Data Cyber Security Analytics systems," *Journal of Network and Computer Applications*, vol. 198, Art. no. 103294, 2021, doi: 10.1016/j.jnca.2021.103294.



[125] M. Haggag, M. M. Tantawy, and M. S. El-Soudani, "Implementing a Deep Learning Model for Intrusion Detection on Apache Spark Platform," *IEEE Access*, vol. 8, pp. 163660–163672, 2020, doi: 10.1109/access.2020.3019931.

[126] A. Imteaj, U. Thakker, S. Wang, J. Li, and M. H. Amini, "A Survey on Federated Learning for Resource-Constrained IoT Devices," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 1–24, 2021, doi: 10.1109/jiot.2021.3095077.

[127] J. Verbraeken, M. Wolting, J. Katzy, J. Kloppenburg, T. Verbelen, and J. S. Rellermeyer, "A Survey on Distributed Machine Learning," *ACM Computing Surveys*, vol. 53, no. 2, pp. 1–33, 2020, doi: 10.1145/3377454.

Dr. M. Rajasekhara Babu received the B.Tech. Degree in Electronics and Communication Engineering from Sri Venkateswara (S.V.) University, Andhra Pradesh, the M.Tech. degree in Computer Science and Engineering from REC Calicut (now NIT Calicut), Kerala, and the Ph.D. degree from VIT University, Vellore, India.

He worked with Intel E.C., India from 2000 to 2002 before joining VIT University in 2002, where he currently serves as a Professor in the School of Computer Science and Engineering. He was instrumental in establishing advanced laboratories in Multi-Core Architectures, Embedded Systems, and Internet of Things (IoT) with support from Intel India. He has collaborated with NUS, NTU Singapore, and the University of Malaya, Malaysia. He has published 10+ patents, guided around 10+ Ph.D. scholars, authored 100+ research publications, co-authored books, and edited international conference proceedings published by Allied Publishers, Macmillan, and Springer. His research interests include IoT, High-Performance Computing (HPC) Artificial Intelligence, and Data Analytics.

Prof. Rajasekhara Babu is associated with the Computer Society of India (CSI), IEEE, and IAENG. He has held several academic and administrative leadership positions at VIT and received the Best Teacher Award (2005-2006) and the Best E-Content Development Award (2007) for his contributions to teaching, research, and academic excellence.

[[STUDENT NAME]] (Reg. No. 26MML0054) is a postgraduate student in the School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore, India. [[Add research interests, contribution to this work and a 1 in x 1.25 in photograph.]]

[[STUDENT NAME]] (Reg. No. 26MML0006) is a postgraduate student in the School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore, India. [[Add research interests, contribution to this work and a 1 in x 1.25 in photograph.]]

[[STUDENT NAME]] (Reg. No. 26MML0047) is a postgraduate student in the School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore, India. [[Add research interests, contribution to this work and a 1 in x 1.25 in photograph.]]

[[STUDENT NAME]] (Reg. No. 26MML0015) is a postgraduate student in the School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore, India. [[Add research interests, contribution to this work and a 1 in x 1.25 in photograph.]]